

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_b3a30c42-346\agent-  
a63d3ec22f9e87198.jsonl`  
- SHA-256: `f7d3d93bb388647b42abad1f9a37e6bd92c10fd456f45469a901c8f959c16eba`  
- Source modified: `2026-06-16T02:28:01+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_b3a30c42-346`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T02:24:52.616Z)

You are mapping the EXISTING regression/golden/eval infrastructure in this badminton rally-detection repo so a design can BUILD ON it (not reinvent). Read these files in full and trace how they fit together: backend/eval/regression.py, backend/eval/gt_loader.py, backend/eval/fusion_golden.py, backend/eval/export_wasb_segments.py, backend/eval/harness.py, backend/eval/metrics.py, backend/eval/segmentation_metrics.py, backend/eval/served_gate_a.py, backend/eval/splits.py, backend/eval/ablation.py, and their tests (tests/test_eval_regression.py, tests/test_gt_loader.py, tests/test_fusion_golden_telemetry.py, tests/test_eval_harness.py, tests/test_served_gate_a_regression.py, tests/test_export_wasb_segments.py). Grep for how these are invoked (CLI entrypoint? pytest only?). ANSWER PRECISELY: (1) What regression/golden capability EXISTS today and how is it run? (2) Does any of it ALREADY run WITHOUT a video present ? i.e. consume committed structured artifacts (CSV/JSON) instead of re-running the detector on video? (3) What exact artifacts does each module consume and produce (file paths, formats, columns)? (4) Is there already a committed 'golden' fixture in the repo and where? (5) For a 'video-free regression from committed structured files' design, what EXISTS to reuse vs what must be ADDED? Use concrete paths and function names.

2. assistant (2026-06-16T02:24:55.806Z)

I'll map the existing eval/regression/golden infrastructure. Let me start by pulling the repo fresh and reading the target files in parallel.

3. user (2026-06-16T02:24:59.912Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-16T02:24:59.941Z)

```
1      """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008  
prereq).  
2  
3      The repo historically carried two incompatible loaders:  
4      - ``annotations.load_annotations``: positional ``start,end`` (the collector's  
5        ``curate export`` format), raises on bad rows;  
6      - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``  
7        (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.  
8  
9      A file in one convention fed to the other loader yielded zero/garbage intervals with no  
10     error. This module accepts BOTH conventions (header-mapped, else positional), so every  
11     quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)  
12     flows through one reader. Both legacy entry points now delegate here.  
13  
14     Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):  
15     - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
```

```

16         columns ? rally_number, ending_reason, sport, ? ? are ignored);
17     - no header: columns 0,1 positionally.
18     Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19     (``annotations.parse_timestamp``).
20     """
21     import csv
22     import logging
23     from typing import List, Optional, Tuple
24     from backend.eval.annotations import parse_timestamp
25
26     logger = logging.getLogger(__name__)
27
28     Interval = Tuple[float, float]
29
30     _START_NAMES = ("start", "start_time")
31     _END_NAMES = ("end", "end_time")
32
33
34     def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35         """If `row` is a header naming start/end columns, return their indices, else
36         None."""
37         cells = [(c or "").strip().lower() for c in row]
38         start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39         end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40         if start_idx is not None and end_idx is not None:
41             return start_idx, end_idx
42         return None
43
44     def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45         """Load GT rally intervals from either golden-CSV convention.
46
47         strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48         (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49         strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50         skipped ? a silently shrinking GT set flatters precision.
51
52         Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53         (they break one-to-one matching and usually indicate a labeling error).
54         """
55         intervals: List[Interval] = []
56         skipped = 0
57         cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
58         saw_header = False
59
60         with open(csv_path, newline="", encoding="utf-8-sig") as f:
61             for lineno, row in enumerate(csv.reader(f), start=1):
62                 if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                     continue
64                 if cols is None and not saw_header:
65                     header = _header_columns(row)
66                     if header is not None:
67                         cols = header
68                         saw_header = True
69                         continue # consume the header row
70                 cols = (0, 1) # headerless: positional start,end
71                 assert cols is not None # every branch above assigns it
72                 si, ei = cols
73                 try:
74                     if len(row) <= max(si, ei):
75                         raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
76 {row!r}")
77                     start = parse_timestamp(row[si])
78                     end = parse_timestamp(row[ei])

```

```

78         if start < 0:
79             raise ValueError(f"negative start ({start})")
80         if start >= end:
81             raise ValueError(f"start ({start}) >= end ({end})")
82     except ValueError as e:
83         if strict:
84             raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85         skipped += 1
86         continue
87     intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
91 skipped)
92
93     intervals.sort(key=lambda iv: iv[0])
94     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
95         if s2 < e1:
96             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
97 "labeling error? Overlaps break one-to-one matching.",
98 csv_path, s1, e1, s2, e2)
99     return intervals

```

5. user (2026-06-16T02:24:59.941Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7     ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8     predictions  ? the production pipeline's stored DB rows    [backend.eval.harness]
9     comparison   ? interval matching + P/R/F1                  [backend.eval.metrics]
10    baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16    """
17    import os
18    import json
19    import hashlib
20    import logging
21    from dataclasses import dataclass, asdict
22    from typing import Dict, List, Optional
23
24    from backend.infrastructure.database import Database
25    from backend.eval import metrics
26    from backend.eval.harness import get_predictions
27    from backend.annotations import annotation_registry
28
29    logger = logging.getLogger(__name__)
30
31    # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32    # actually load a committed baseline to compare against.
33    DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34    DEFAULT_TOLERANCE = 0.05
35
36
37    def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38        """Stable short hash of the GT intervals ? detects a baseline taken against
39        different labels."""

```

```

39         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                      f1: float, path: str = DEFAULT_BASELINE_PATH,
82                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                      gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
detected
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)
100

```

```

101
102 # ----- the runner
103
104 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105            stage: str = "final", iou_threshold: float = 0.5,
106            detector: Optional[str] = None,
107            tolerance: float = DEFAULT_TOLERANCE,
108            baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109     """Score stored predictions for one video against ground truth and compare to
baseline.
110
111     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113     precision OR recall falls more than `tolerance` below the snapshotted baseline.
114     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115     caller can choose to snapshot the current numbers via `save_baseline`.
116     """
117     preds = get_predictions(db, video_id, stage, detector=detector)
118     s = metrics.score(preds, ground_truth, iou_threshold)
119     fp = gt_hash(ground_truth)
120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148     else:
149         reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151     return RegressionResult(
152         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153         precision=s.precision, recall=s.recall, f1=s.f1,
154         baseline_precision=base_p, baseline_recall=base_r,
155         tolerance=tolerance, regressed=regressed, reasons=reasons,
156         has_baseline=has_baseline, gt_hash=fp,

```

```

157         )
158
159
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                             tier: str = "file", stage: str = "final",
162                             iou_threshold: float = 0.5,
163                             detector: Optional[str] = None,
164                             tolerance: float = DEFAULT_TOLERANCE,
165                             baseline_path: str = DEFAULT_BASELINE_PATH,
166                             guideline: Optional[str] = None,
167                             sport: str = "badminton",
168                             provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + ".join(f'\n    - {why}' for why in r.reasons)
189

```

6. user (2026-06-16T02:24:59.947Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature
columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse

```

```

20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32
33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
42         convention)."""
43         if spec.get("video_path"):
44             return str(spec["video_path"])
45         labels = str(spec["labels"])
46         if labels.endswith(".rallies.csv"):
47             return labels[: -len(".rallies.csv")] + ".MP4"
48         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
49         *.rallies.csv")
50
51     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
52                      frame_skip: int = 3, iou: float = 0.5,
53                      log_every_sec: float = 15.0,
54                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
55         """One golden video -> a feature record (shuttle + person features per candidate
56         window).
57
58         Returns `{name, fps, frame_width,
59         candidates:[{start,end,shuttle{ },person{ },label}], gts}`.
60         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
61         tests;
62
63         when None a real `PersonDetector` is built (loads torchvision on first use).
64         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
65         snapshots
66
67         from the detection progress callback so a hard kill leaves a forensic runlog.
68         """
69         fps, fw = float(spec["fps"]), float(spec["frame_width"])
70         traj = str(spec["trajectory"])
71         points = TrackNetRunner.parse_trajectory_csv(traj)
72         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
73         gts = load_golden_gts(str(spec["labels"]))
74         labels = label_candidates(intervals, gts, iou)
75         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
76         estimate
77         # (parity with the shuttle net-crossing gate) rather than hard-coding an
78         orientation.
79         axis, net_px, _span = rally_gate.estimate_play_axis(points)
80         video_path = _derive_video_path(spec)
81
82         if detector is None:
83             from backend.pipeline.detectors.person_detector import PersonDetector
84             detector = PersonDetector({"indexing": {"use_gpu": True}})

```

```

77     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79     n = len(intervals)
80     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81     print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82     t0 = time.monotonic()
83     state = {"last": t0}
84
85     def _progress(done: int, total: int) -> None:
86         if telemetry is not None:
87             telemetry.snapshot(done, total, phase=spec["name"])
88         now = time.monotonic()
89         if now - state["last"] >= log_every_sec or done >= total:
90             el = now - t0
91             rate = done / el if el > 0 else 0.0
92             eta = (total - done) / rate if rate > 0 else 0.0
93             pct = 100 * done // total if total else 0
94             print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                   f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96             state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                    progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                            court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),
121         })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
last);
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
136             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])))

```



```

137                                     if os.path.isfile(s.get("trajectory", ""))
else 0))
138         os.makedirs(out_dir, exist_ok=True)
139         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
140         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
141         # break extraction, so guard the build and fall back to no telemetry on any error.
142         telem: Optional[RunTelemetry]
143         try:
144             telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"),
label="fusion_golden")
145         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
146             telem = None
147         written: List[str] = []
148         for vi, spec in enumerate(specs, 1):
149             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
150             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
151             # run left off instead of re-detecting). --fresh forces a redo.
152             if not fresh and os.path.isfile(out):
153                 print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
154                     f"-> {out} (use --fresh to redo)", flush=True)
155                 written.append(out)
156                 continue
157             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
158             t0 = time.monotonic()
159             if telem is not None:
160                 telem.event("video_start", spec["name"])
161             try:
162                 data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
163             except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
164                 if telem is not None:
165                     telem.event("error", spec["name"], err=str(e))
166                 raise
167             n_pos = sum(c["label"] for c in data["candidates"])
168             with open(out, "w", encoding="utf-8") as f:
169                 json.dump(data, f, indent=2)
170             if telem is not None:
171                 telem.event("video_done", spec["name"],
172                             windows=len(data["candidates"]), positives=n_pos)
173             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
174                 f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
175                 f"-> {out}", flush=True)
176             written.append(out)
177         return written
178
179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188                         help="process smaller videos first for quick feedback (LOVO is

```

```

order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193
194     if __name__ == "__main__":
195         main()
196

```

7. assistant (2026-06-16T02:25:04.222Z)

Let me read the remaining target files and tests in parallel.

8. user (2026-06-16T02:25:05.190Z)

```

1     """Export tuned WASB rally segments for manual eyeball verification.
2
3     After calibration picks a tuned threshold config, this turns a cached WASB
4     trajectory into the actual rally [start,end] segments that config produces, so the
5     user can answer "are the rallies actually better?" ? by scrubbing the video to each
6     segment, by diffing tuned-vs-golden quantitatively, or by cutting clips.
7
8     Reuses (no new windowing/metrics/cutting logic):
9     - calibrate_wasb.make_predict_fn -> trajectory -> segments for a given config
10    - eval.metrics.match_intervals/score -> tuned-vs-golden IoU comparison
11    - tools.rally_slicer.slice_rallies -> optional clip cutting (the exported CSV is
12      written in the golden/slicer schema, so it feeds straight back in)
13
14    Run (uses the tuned config from the first calibration by default):
15    python -m backend.eval.export_wasb_segments --trajectory
~/clips/sample_long_traj.csv \
16        --labels "Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-
width 1920
17    # add --video <mp4> --slice to also cut one padded clip per tuned rally
18    """
19    from __future__ import annotations
20
21    import csv
22    import os
23    import os.path as osp
24    import argparse
25    from typing import List, Tuple, Dict, Optional
26
27    from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
28    from backend.eval.metrics import match_intervals, score
29    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30    from backend.pipeline.detectors import rally_gate
31
32    Interval = Tuple[float, float]
33    Config = Tuple[float, float, float]
34
35    # The tuned config from the first per-video calibration (velocity, merge_gap, min_dur);
36    # F1 0.588 (baseline) -> 0.746. Override on the CLI to export a different operating
point.
37    TUNED: Config = (0.05, 1.0, 1.0)
38
39    SEG_FIELDS = ["rally_number", "start_time", "end_time", "duration", "start_mmss",
"end_mmss"]
40    COMP_FIELDS = ["type", "ref", "start_time", "end_time", "mmss", "status",
41                  "iou", "pred_start", "pred_end", "start_err_s", "end_err_s"]
42
43
44    def seconds_to_mmss(t: float) -> str:
45        """Human-scrubbable timestamp, e.g. 106.031 -> '1:46.031'."""

```

```

46         t = max(0.0, float(t))
47         m = int(t // 60)
48         return f"{m}:{t - m * 60:06.3f}"
49
50
51 def segments_to_rows(segments: List[Interval]) -> List[Dict[str, str]]:
52     """Number segments 1..N in time order; emit the golden/slicer CSV schema."""
53     rows = []
54     for i, (s, e) in enumerate(segments, start=1):
55         rows.append({
56             "rally_number": str(i),
57             "start_time": f"{s:.3f}",
58             "end_time": f"{e:.3f}",
59             "duration": f"{e - s:.3f}",
60             "start_mmss": seconds_to_mmss(s),
61             "end_mmss": seconds_to_mmss(e),
62         })
63     return rows
64
65
66 def _ensure_parent(path: str) -> None:
67     os.makedirs(osp.dirname(osp.abspath(path)) or ".", exist_ok=True)
68
69
70 def write_segments_csv(path: str, segments: List[Interval]) -> str:
71     """Write segments in the golden schema (rally_number/start_time/end_time + mm:ss).
72
73     The schema is what tools.rally_slicer.load_rally_labels consumes, so the file
74     doubles as input to clip cutting.
75     """
76     _ensure_parent(path)
77     with open(path, "w", newline="", encoding="utf-8") as f:
78         w = csv.DictWriter(f, fieldnames=SEG_FIELDS)
79         w.writeheader()
80         w.writerows(segments_to_rows(segments))
81     return path
82
83
84 def build_comparison(preds: List[Interval], gts: List[Interval],
85                     iou_threshold: float = 0.5) -> dict:
86     """Golden-centric comparison: per golden rally, did a tuned segment match (IoU),
87     plus the boundary errors and any extra (false-positive) tuned segments."""
88     mr = match_intervals(preds, gts, iou_threshold)
89     sc = score(preds, gts, iou_threshold, match_result=mr)
90     by_gt = {m.gt_index: m for m in mr.matches}
91
92     rows: List[Dict[str, str]] = []
93     for gi, (gs, ge) in enumerate(gts, start=0):
94         m = by_gt.get(gi)
95         if m is not None:
96             ps, pe = preds[m.pred_index]
97             rows.append({
98                 "type": "golden", "ref": str(gi + 1),
99                 "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
100                 "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
101                 "status": "matched", "iou": f"{m.iou:.3f}",
102                 "pred_start": f"{ps:.3f}", "pred_end": f"{pe:.3f}",
103                 "start_err_s": f"{ps - gs:+.3f}", "end_err_s": f"{pe - ge:+.3f}",
104             })
105         else:
106             rows.append({
107                 "type": "golden", "ref": str(gi + 1),
108                 "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
109                 "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
110                 "status": "MISS", "iou": "0.000",

```

```

111         "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
112     })
113     for pi in mr.unmatched_pred_indices: # extra tuned segments = false
positives
114         ps, pe = preds[pi]
115         rows.append({
116             "type": "pred_extra", "ref": str(pi + 1),
117             "start_time": f"{ps:.3f}", "end_time": f"{pe:.3f}",
118             "mmss": f"{seconds_to_mmss(ps)}-{seconds_to_mmss(pe)}",
119             "status": "false_positive", "iou": "",
120             "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
121         })
122     return {"score": sc, "rows": rows}
123
124
125     def write_comparison_csv(path: str, comparison: dict) -> str:
126         _ensure_parent(path)
127         with open(path, "w", newline="", encoding="utf-8") as f:
128             w = csv.DictWriter(f, fieldnames=COMP_FIELDS)
129             w.writeheader()
130             w.writerows(comparison["rows"])
131         return path
132
133
134     def _with_suffix(path: str, suffix: str) -> str:
135         root, ext = osp.splitext(path)
136         return root + suffix + ext
137
138
139     def run(trajecory_csv: str, fps: float, frame_width: float, cfg: Config = TUNED,
140            out_csv: str = "output/wasb_tuned_segments.csv", labels_csv: Optional[str] =
None,
141            iou: float = 0.5, baseline: bool = False,
142            comparison_out: Optional[str] = None, min_crossings: int = 0) -> dict:
143         predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
144         # Gate A: drop between-point false fires (single service-feed tosses) by requiring
145         # the shuttle to cross the net >= min_crossings times within the window.
146         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv) if min_crossings > 0
else None
147
148         def _gate(segs: List[Interval], label: str) -> List[Interval]:
149             if not points:
150                 return segs
151             kept = rally_gate.apply_gate(points, segs, fps, min_crossings=min_crossings)
152             print(f"[gate] {label}: {len(segs)} -> {len(kept)} segments "
153                   f"(removed {len(segs) - len(kept)} with <{min_crossings} net-crossings)")
154             return kept
155
156         tuned_segs = _gate(predict_fn(cfg), "tuned")
157         write_segments_csv(out_csv, tuned_segs)
158         total = sum(e - s for s, e in tuned_segs)
159         print("=== WASB tuned-segment export ===")
160         print(f"tuned cfg {cfg}: {len(tuned_segs)} rally segments, {total:.1f}s of play ->
{out_csv}")
161         result: dict = {"tuned_cfg": cfg, "num_tuned": len(tuned_segs), "out_csv": out_csv}
162
163         if baseline:
164             base_segs = _gate(predict_fn(BASELINE), "baseline")
165             base_out = _with_suffix(out_csv, "_baseline")
166             write_segments_csv(base_out, base_segs)
167             print(f"baseline cfg {BASELINE}: {len(base_segs)} segments -> {base_out}")
168             result["num_baseline"] = len(base_segs)
169             result["baseline_csv"] = base_out
170
171         if labels_csv:

```

```

172         gts = load_golden_gts(labels_csv)
173         if not gts:
174             raise SystemExit(f"no usable golden rallies in {labels_csv}")
175         comp = build_comparison(tuned_segs, gts, iou)
176         sc = comp["score"]
177         comp_out = comparison_out or _with_suffix(out_csv, "_vs_golden")
178         write_comparison_csv(comp_out, comp)
179         print(f"\n--- tuned vs golden ({len(gts)} rallies, IoU>={iou}) ---")
180         print(f"matched (TP) {sc.true_positives} | misses (FN) {sc.false_negatives} | "
181               f"extra (FP) {sc.false_positives}")
182         print(f"precision {sc.precision:.3f} | recall {sc.recall:.3f} | f1 {sc.f1:.3f} | "
183               f"meanIoU {sc.mean_iou:.3f}")
184         print(f"mean boundary error: start {sc.mean_start_error:.2f}s | end "
185               f"{sc.mean_end_error:.2f}s")
186         print(f"per-rally detail -> {comp_out}")
187         result["comparison"] = sc.as_dict()
188         result["comparison_csv"] = comp_out
189         return result
190
191     def _maybe_slice(video: str, segments_csv: str, clips_dir: str) -> None:
192         """Cut one padded clip per exported rally (reuses the rally slicer)."""
193         from backend.tools.rally_slicer import slice_rallies
194         written = slice_rallies(video, segments_csv, ["all"], clips_dir)
195         print(f"\ncut {len(written)} clip(s) -> {clips_dir}")
196
197     def main() -> None:
198         ap = argparse.ArgumentParser(description="Export tuned WASB rally segments for "
199               "eyeballing.")
200         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV "
201               "(Frame,Visibility,X,Y)")
202         ap.add_argument("--fps", type=float, default=59.94)
203         ap.add_argument("--frame-width", type=float, default=1920.0)
204         ap.add_argument("--velocity-thresh", type=float, default=TUNED[0])
205         ap.add_argument("--merge-gap", type=float, default=TUNED[1])
206         ap.add_argument("--min-duration", type=float, default=TUNED[2])
207         ap.add_argument("--labels", default=None, help="golden labels CSV -> tuned-vs-golden "
208               "comparison")
209         ap.add_argument("--iou", type=float, default=0.5)
210         ap.add_argument("--out", default="output/wasb_tuned_segments.csv")
211         ap.add_argument("--comparison-out", default=None)
212         ap.add_argument("--baseline", action="store_true", help="also export baseline-config "
213               "segments")
214         ap.add_argument("--min-crossings", type=int, default=0,
215               help="gate A: drop windows whose shuttle crosses the net < N times "
216               "(kills single-toss service-feed false fires; 0=off, 2 "
217               "recommended)")
218         ap.add_argument("--video", default=None, help="source video (with --slice) to cut "
219               "clips from")
220         ap.add_argument("--slice", action="store_true", help="cut one padded clip per tuned "
221               "rally")
222         ap.add_argument("--clips-dir", default="output/wasb_tuned_clips")
223         args = ap.parse_args()
224
225         cfg = (args.velocity_thresh, args.merge_gap, args.min_duration)
226         res = run(args.trajectory, args.fps, args.frame_width, cfg=cfg, out_csv=args.out,
227               labels_csv=args.labels, iou=args.iou, baseline=args.baseline,
228               comparison_out=args.comparison_out, min_crossings=args.min_crossings)
229         if args.slice:
230             if not args.video:
231                 raise SystemExit("--slice requires --video")
232             _maybe_slice(args.video, res["out_csv"], args.clips_dir)

```

```

228
229     if __name__ == "__main__":
230         main()
231

```

9. user (2026-06-16T02:25:06.166Z)

```

1     """Score the pipeline's detected rallies against ground-truth annotations.
2
3     Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
4     so scoring is fast, deterministic, and free. Two prediction "stages" can be
5     scored:
6
7     - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
8       the shuttle/person detector found, before AI confirmation.
9     - ``final`` ? confirmed play segments (``play_segments`` table). After the
10      AI handoff trimmed/validated boundaries.
11
12     Comparing the two tells you whether the weak link is *detection* (candidates
13     already miss rallies) or *segmentation* (candidates catch them but final is
14     wrong) ? see docs/EVALUATION.md.
15     """
16
17     import logging
18     from dataclasses import dataclass, field
19     from typing import Any, List, Dict, Optional
20
21     from backend.infrastructure.database import Database
22     from backend.eval import metrics
23     from backend.eval.metrics import Interval, Score, MatchResult
24
25     logger = logging.getLogger(__name__)
26
27     STAGES = ("candidates", "final")
28
29
30     @dataclass
31     class StageReport:
32         stage: str
33         pred_intervals: List[Interval]
34         score: Score
35         match_result: MatchResult
36         pr_curve: List[Score] = field(default_factory=list)
37
38
39     @dataclass
40     class EvalReport:
41         video_id: str
42         iou_threshold: float
43         gt_intervals: List[Interval]
44         stages: Dict[str, StageReport] = field(default_factory=dict)
45
46
47     def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
48         """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
49         ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
50         ivs.sort(key=lambda iv: iv[0])
51         return ivs
52
53
54     def get_predictions(db: Database, video_id: str, stage: str,
55                        detector: Optional[str] = None) -> List[Interval]:
56         """Fetch predicted rally intervals for one stage from the DB."""
57         if stage == "candidates":
58             rows = db.query_candidate_segments(video_id, detector=detector)

```

```

59     elif stage == "final":
60         rows = db.query_play_segments(video_id, {})
61     else:
62         raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
63     return _rows_to_intervals(rows)
64
65
66 def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
67             stages: List[str], iou_threshold: float = 0.5,
68             detector: Optional[str] = None,
69             with_pr_curve: bool = False) -> EvalReport:
70     """Score the requested stages for one video against ground truth."""
71     report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
72                        gt_intervals=gt_intervals)
73     for stage in stages:
74         preds = get_predictions(db, video_id, stage, detector=detector)
75         mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
76         s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
77         curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
78         report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
79                                           score=s, match_result=mr, pr_curve=curve)
80     return report
81
82
83 # ----- reporting
84
85 def _fmt_ts(seconds: float) -> str:
86     seconds = max(0, int(seconds))
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     return f"{h:02d}:{m:02d}:{s:02d}"
90
91
92 def report_to_dict(report: EvalReport) -> dict:
93     """JSON-serialisable view of an EvalReport."""
94     out: Dict[str, Any] = {
95         "video_id": report.video_id,
96         "iou_threshold": report.iou_threshold,
97         "num_ground_truth": len(report.gt_intervals),
98         "stages": {},
99     }
100     for stage, sr in report.stages.items():
101         out["stages"][stage] = {
102             "score": sr.score.as_dict(),
103             "pr_curve": [s.as_dict() for s in sr.pr_curve],
104         }
105     return out
106
107
108 def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109     """Human-readable console report for one video."""
110     lines = [
111         "=" * 64,
112         f" Rally evaluation ? video '{report.video_id}'",
113         f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
114         "=" * 64,
115         f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116         f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117         "-" * 64,
118     ]
119     for stage in report.stages:
120         s = report.stages[stage].score
121         lines.append(
122             f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}

```

```

"
123         f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124         f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125     )
126     lines.append("=" * 64)
127
128     if show_failures:
129         for stage, sr in report.stages.items():
130             misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
131             fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132             lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
133             for iv in misses:
134                 lines.append(f"    - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135             lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136             for iv in fps:
137                 lines.append(f"    - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139     return "\n".join(lines)
140

```

10. user (2026-06-16T02:25:06.637Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19        overlap / union, both measured as durations on the timeline.
20        """
21        a_start, a_end = a
22        b_start, b_end = b
23        if a_end <= a_start or b_end <= b_start:
24            return 0.0
25        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26        if inter <= 0.0:
27            return 0.0
28        union = (a_end - a_start) + (b_end - b_start) - inter
29        return inter / union if union > 0 else 0.0
30
31
32    @dataclass
33    class Match:
34        """One matched predicted?ground-truth pair."""
35        pred_index: int
36        gt_index: int
37        iou: float
38
39
40    @dataclass

```



```

41     class MatchResult:
42         """Outcome of matching predictions to ground truth at a given IoU threshold."""
43         matches: List[Match] = field(default_factory=list)
44         unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45         unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
46     (misses)
47
48     def match_intervals(preds: List[Interval], gts: List[Interval],
49                       iou_threshold: float = 0.5) -> MatchResult:
50         """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52         Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53         are claimed highest-IoU first, and each prediction/GT can be used once. This
54         keeps a single prediction from being credited for two ground-truth rallies
55         (and vice versa).
56         """
57         candidates = []
58         for pi, p in enumerate(preds):
59             for gi, g in enumerate(gts):
60                 iou = interval_iou(p, g)
61                 # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
62                 # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63                 if iou > 0.0 and iou >= iou_threshold:
64                     candidates.append((iou, pi, gi))
65                 # Highest IoU first; ties broken by indices for determinism.
66                 candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68         used_pred, used_gt = set(), set()
69         matches: List[Match] = []
70         for iou, pi, gi in candidates:
71             if pi in used_pred or gi in used_gt:
72                 continue
73             used_pred.add(pi)
74             used_gt.add(gi)
75             matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77         unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78         unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79         return MatchResult(matches=matches,
80                           unmatched_pred_indices=unmatched_pred,
81                           unmatched_gt_indices=unmatched_gt)
82
83
84     @dataclass
85     class Score:
86         """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87         iou_threshold: float
88         num_pred: int
89         num_gt: int
90         true_positives: int
91         false_positives: int
92         false_negatives: int
93         precision: float
94         recall: float
95         f1: float
96         mean_iou: float # over matched pairs (0.0 if none)
97         mean_start_error: float # mean |pred_start - gt_start| over matches, seconds
98         mean_end_error: float # mean |pred_end - gt_end| over matches, seconds
99
100         def as_dict(self) -> dict:
101             return self.__dict__.copy()
102
103

```

```

104     def score(preds: List[Interval], gts: List[Interval],
105               iou_threshold: float = 0.5,
106               match_result: Optional[MatchResult] = None) -> Score:
107         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110         extracting the failure lists); otherwise matching is done here.
111         """
112         mr = match_result if match_result is not None else match_intervals(preds, gts,
113 iou_threshold)
114         tp = len(mr.matches)
115         fp = len(mr.unmatched_pred_indices)
116         fn = len(mr.unmatched_gt_indices)
117
118         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
119         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
120         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
121     else 0.0
122
123     if mr.matches:
124         mean_iou = sum(m.iou for m in mr.matches) / tp
125         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
126 mr.matches) / tp
127         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
128 mr.matches) / tp
129     else:
130         mean_iou = mean_start_err = mean_end_err = 0.0
131
132     return Score(
133         iou_threshold=iou_threshold,
134         num_pred=len(preds), num_gt=len(gts),
135         true_positives=tp, false_positives=fp, false_negatives=fn,
136         precision=precision, recall=recall, f1=f1,
137         mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
138     )
139
140     def pr_curve(preds: List[Interval], gts: List[Interval],
141                 thresholds: Optional[List[float]] = None) -> List[Score]:
142         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
143
144         Lets you see strict-vs-lenient behaviour rather than a single operating point.
145         """
146         if thresholds is None:
147             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
148         return [score(preds, gts, iou_threshold=t) for t in thresholds]

```

11. user (2026-06-16T02:25:06.885Z)

```

1     """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3     Temporal IoU (`metrics.score`) is nearly blind to *fragmentation*: a single rally
4     chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5     still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6     literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7     thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8     sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9     tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10    structure invisibly. See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C4.
11
12    This is **additive**: new functions over the existing ``metrics`` matcher; nothing here
13    changes ``score``/``compare``/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
14    ?

```

```

14     so it runs in the GPU-free / numpy-free lanes too.
15
16     Conventions match ``metrics``: an ``Interval`` is a ``(start, end)`` tuple of seconds.
17     A ``ScoredInterval`` is ``(interval, confidence)`` ? the confidence is whatever the
scorer
18     emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.
19
20     Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
the
21     ordered sequence of segment *labels* and is degenerate for a single action class (every
22     rally carries the same label), so it is intentionally omitted ? F1@k + mAP@tIoU + the
23     segment-count ratio are the over-segmentation-sensitive metrics for this binary setting.
24     """
25     from __future__ import annotations
26
27     from typing import Dict, List, Sequence, Tuple
28
29     from backend.eval.metrics import Interval, interval_iou, score
30
31     #: A predicted interval plus the confidence used to rank it for average precision.
32     ScoredInterval = Tuple[Interval, float]
33
34     #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
35     DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
36     #: Default tIoU sweep for mean average precision ? the action-detection convention.
37     DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)
38
39
40     def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
41                       overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
42         """Segmental F1 at each overlap threshold (``{overlap: f1}``) ? the TAS F1@k.
43
44         Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
45         for 1 rally) scores extra false positives and a merged prediction misses a rally ?
46         both drag F1 down where temporal-IoU alone would not flinch.
47         """
48         return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}
49
50
51     def average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
52                          iou_threshold: float = 0.5) -> float:
53         """Average precision at a single tIoU (all-points / VOC-2010 interpolation).
54
55         Predictions are ranked by descending confidence; each is a true positive iff it
56         claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
57         available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
58         there are no ground-truth rallies or no predictions reach the threshold.
59         """
60         n_gt = len(gts)
61         if n_gt == 0 or not scored_preds:
62             return 0.0
63
64         order = sorted(range(len(scored_preds)),
65                       key=lambda i: (-scored_preds[i][1], i)) # conf desc, stable by
index
66         matched_gt = set()
67         precisions: List[float] = []
68         recalls: List[float] = []
69         cum_tp = 0
70         cum_fp = 0
71         for i in order:
72             interval, _conf = scored_preds[i]
73             best_iou, best_gi = 0.0, -1
74             for gi, g in enumerate(gts):
75                 if gi in matched_gt:

```

```

76         continue
77         iou = interval_iou(interval, g)
78         if iou >= iou_threshold and iou > best_iou:
79             best_iou, best_gi = iou, gi
80     if best_gi >= 0:
81         matched_gt.add(best_gi)
82         cum_tp += 1
83     else:
84         cum_fp += 1
85     precisions.append(cum_tp / (cum_tp + cum_fp))
86     recalls.append(cum_tp / n_gt)
87
88     # Monotone precision envelope (take the max precision to the right), then integrate
89     # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
90     for i in range(len(precisions) - 2, -1, -1):
91         precisions[i] = max(precisions[i], precisions[i + 1])
92     ap = 0.0
93     prev_recall = 0.0
94     for p, r in zip(precisions, recalls):
95         if r > prev_recall:
96             ap += p * (r - prev_recall)
97             prev_recall = r
98     return ap
99
100
101 def mean_average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
102                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
103     """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
104     tious = list(iou_thresholds)
105     if not tious:
106         return 0.0
107     return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)
108
109
110 def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
111     """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.
112
113     ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
114     merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
115     ``gts``
116
117     is empty (use F1@k / mAP for the graded picture).
118     """
119     if not gts:
120         return 0.0
121     return len(preds) / len(gts)
122
123
124 def segmentation_report(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
125                         overlaps: Sequence[float] = DEFAULT_OVERLAPS,
126                         iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
127 object]:
128     """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio.
129
130     ``scored_preds`` carry confidences for AP; F1@k uses the bare intervals. Returns a
131     plain dict (JSON-friendly, string keys) so it can ride next to a ``compare`` result
132     or land in the experiment leaderboard.
133     """
134     preds = [iv for iv, _conf in scored_preds]
135     return {
136         "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
137 overlaps).items()},
138         "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
139 iou_thresholds},
140         "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
141         "segment_count_ratio": segment_count_ratio(preds, gts),

```

```

137         "num_pred": len(preds),
138         "num_gt": len(gts),
139     }
140

```

12. assistant (2026-06-16T02:25:09.807Z)

Now let me read the remaining target source files and check the git branch situation.

13. user (2026-06-16T02:25:10.341Z)

```

1     """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seams
2     on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.
3
4     The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on
the
5     **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-
scored
6     through the experiment harness's ``_gate_mask``. That validates the *signal*. It does
NOT
7     exercise the **served path**: in production the windows come from
8     ``trajectory_to_action_windows`` at the production operating point
9     (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),
the
10    net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the
gate is
11    ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production
methods ?
12    no reimplementaion of the windowing or the gate ? on a cached trajectory CSV, so the
13    verification reflects what ``process_video`` actually hands to Gemini.
14
15    GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
WOULD
16    be sent to the AI). With the person cue OFF (the default) nothing here loads torch or
touches
17    the GPU ? it just re-scores cached WASB trajectories.
18
19    Run (owner box, golden trajectories present):
20    python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
21    """
22    from __future__ import annotations
23
24    import argparse
25    import json
26    from dataclasses import dataclass
27    from typing import Any, Dict, List, Optional, Tuple
28
29    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30    from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
31    from backend.eval.calibrate_wasb import load_golden_gts
32    from backend.eval.metrics import Interval, score, Score
33    from backend.eval import segmentation_metrics as _segm
34
35    # Production windowing operating point (config.json indexing defaults ? the values
36    # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
37    PROD_VELOCITY_THRESH = 0.02
38    PROD_MERGE_GAP = 2.0
39    PROD_MIN_WINDOW = 2.0
40
41
42    def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
43        """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
44        config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
45        person cue OFF (so no GPU / torch is touched)."""
46        return {

```

```

47         "fusion": {
48             "substrate": "wasb",
49             "min_crossings": min_crossings,
50             "min_players": 0,
51             "cue_flags": {},
52         },
53         "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
54         "dead_time_merge_gap": PROD_MERGE_GAP,
55         "min_action_window_duration": PROD_MIN_WINDOW,
56     }
57
58
59     def served_handoff_windows(trajecory_csv: str, fps: float, frame_width: float,
60                               min_crossings: int) -> List[Interval]:
61         """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
62         return the windows that would reach the AI handoff (i.e. the served rally candidates).
63
64         Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
65         (computes ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
66         production ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
67         is OFF.
68
69         """
70         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
71         windows = TrackNetRunner.trajectory_to_action_windows(
72             points, fps=fps, frame_width=frame_width,
73             velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
74             min_window_duration=PROD_MIN_WINDOW,
75         )
76         idx_cfg = _served_idx_cfg(min_crossings)
77         # Construct the real served segmenter (db=None, config carries the indexing block).
78         We only
79         # call the pure decision seams ? no process_video, no DB, no provider ? exactly as
80         the
81         # registry constructs it for `name`/`description`.
82         seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
83         # The served per-window stats (incl. net_crossings) via the production enrichment
84         hook.
85         window_stats = [
86             seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
87                                       idx_cfg=idx_cfg)
88             for cw in windows
89         ]
90         keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
91         return [(windows[i]["start"], windows[i]["end"]) for i in keep]
92
93
94     @dataclass
95     class ServedVideoResult:
96         name: str
97         num_gt: int
98         on: Score          # Gate-A ON (min_crossings=2)
99         off: Score         # Gate-A OFF (min_crossings=0, control)
100         seg_ratio_on: float # |preds| / |gts| with Gate-A ON (over-segmentation)
101         seg_ratio_off: float
102         n_handoff_on: int
103         n_handoff_off: int
104
105         @property
106         def delta_f1(self) -> float:
107             return self.on.f1 - self.off.f1

```

```

105     def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
106                        on_crossings: int = 2) -> ServedVideoResult:
107         """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
windows for
108         one golden video against its golden labels."""
109         fps, fw = float(spec["fps"]), float(spec["frame_width"])
110         traj, labels = str(spec["trajectory"]), str(spec["labels"])
111         gts = load_golden_gts(labels)
112         preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
113         preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
114         return ServedVideoResult(
115             name=str(spec["name"]),
116             num_gt=len(gts),
117             on=score(preds_on, gts, iou),
118             off=score(preds_off, gts, iou),
119             seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
120             seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
121             n_handoff_on=len(preds_on),
122             n_handoff_off=len(preds_off),
123         )
124
125
126     def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
127                        on_crossings: int = 2) -> List[ServedVideoResult]:
128         return [evaluate_video(s, iou, on_crossings) for s in specs]
129
130
131     def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
132         """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
133         n = len(results)
134
135         def _mean(getter) -> float:
136             return sum(getter(r) for r in results) / n if n else 0.0
137
138         deltas = [r.delta_f1 for r in results]
139         wins = sum(1 for d in deltas if d > 1e-9)
140         losses = sum(1 for d in deltas if d < -1e-9)
141         return {
142             "num_videos": n,
143             "mean_f1_on": _mean(lambda r: r.on.f1),
144             "mean_f1_off": _mean(lambda r: r.off.f1),
145             "mean_delta_f1": _mean(lambda r: r.delta_f1),
146             "mean_precision_on": _mean(lambda r: r.on.precision),
147             "mean_precision_off": _mean(lambda r: r.off.precision),
148             "mean_recall_on": _mean(lambda r: r.on.recall),
149             "mean_recall_off": _mean(lambda r: r.off.recall),
150             "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
151             "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
152             "wins": wins,
153             "losses": losses,
154         }
155
156
157     def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
str:
158         lines = [
159             f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU={iou})
===",
160             f"  {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
161             f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
162         ]
163         for r in results:
164             lines.append(
165                 f"  {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
166                 f"{r.delta_f1:>7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "

```

```

167         f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
168     lines += [
169         f" {'-' * 92}",
170         f" MEAN F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on) "
171         f"?F1={agg['mean_delta_f1']:+.3f} "
172         f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f} "
173         f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
174         f" over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
175         f"{agg['mean_seg_ratio_on']:.2f} (on) | wins {agg['wins']}/{agg['num_videos']}"
176         f"(losses {agg['losses']})",
177     ]
178     return "\n".join(lines)
179
180
181     def run(manifest_path: str, iou: float = 0.5) -> Tuple[List[ServedVideoResult],
182 Dict[str, Any]]:
183         with open(manifest_path, encoding="utf-8") as f:
184             specs = json.load(f)
185             results = evaluate_manifest(specs, iou=iou)
186             return results, aggregate(results)
187
188     def main(argv: Optional[List[str]] = None) -> None:
189         ap = argparse.ArgumentParser(
190             description="Verify served Gate-A on real footage via the production
191 FusionSegmenter path.")
192         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
193         ap.add_argument("--iou", type=float, default=0.5)
194         args = ap.parse_args(argv)
195         results, agg = run(args.manifest, iou=args.iou)
196         print(format_table(results, agg, args.iou))
197
198     if __name__ == "__main__":
199         main()
200

```

14. user (2026-06-16T02:25:11.491Z)

```

1     """Train/eval split guard (pure-logic core, in-container).
2
3     Spec from BACKLOG.md "Train/eval separation guardrails" / DEFERRED #13
4     and plan M0:
5     - Split unit = match (never clip / file MD5).
6     - load_split(manifest) -> (train_ids, eval_ids) keyed by match id.
7     - assert_disjoint(train_ids, eval_ids) raises on any overlap.
8     - This would have caught train/eval leakage in training/eval paths.
9     - Cross-repo collector manifest `split` field wiring + wiring into
10     distill_local / run_regression deferred to later PR.
11
12     Usage (once wired):
13         train, eval_ = load_split(manifest)
14         assert_disjoint(train, eval_)
15     """
16     from __future__ import annotations
17
18     import json
19     import os
20     from typing import Any, Dict, Iterable, List, Set, Tuple, Union, cast
21
22
23     def load_split(manifest: Union[str, Iterable[Dict[str, Any]], os.PathLike[str]]) ->
24 Tuple[Set[str], Set[str]]:
25         """Load (train_ids, eval_ids) sets keyed by match id.

```



```

25
26     manifest: path to JSON (str/bytes/PathLike) or iterable of dicts. Each entry should
have 'match_id'
27     (or 'id' fallback; must be non-empty str for real match key; name/video_id
deliberately not used as they are weak/MD5).
28
29     Raises ValueError on:
30     - non-list manifest after load
31     - no real match-level key (empty/falsy after explicit check)
32     - split label not exactly "train" or "eval"
33
34     The collector 'split' field and cross-repo wiring is deferred; this supports
35     synthetic or pre-split manifests for the pure-logic guard and tests.
36     """
37     if isinstance(manifest, (str, bytes, os.PathLike)):
38         with open(manifest, "r", encoding="utf-8-sig") as f:
39             raw = json.load(f)
40             if not isinstance(raw, list):
41                 raise ValueError(f"manifest must be a list of entries, got {type(raw)}")
42             data = cast(List[Dict[str, Any]], raw)
43     else:
44         if isinstance(manifest, dict):
45             raise ValueError(f"manifest must be a list of entries, got
{type(manifest)}")
46         data = list(manifest)
47         if not isinstance(data, list):
48             raise ValueError(f"manifest must be a list of entries, got {type(data)}")
49
50     train_ids: Set[str] = set()
51     eval_ids: Set[str] = set()
52
53     for entry in data:
54         if not isinstance(entry, dict):
55             continue
56         # Explicit not-None + non-empty for real match key; no silent fallback to
name/video_id (weak/MD5)
57         mid = entry.get("match_id")
58         if mid is None or str(mid).strip() == "":
59             mid = entry.get("id")
60         if mid is None or str(mid).strip() == "":
61             raise ValueError(f"no real match-level key (match_id or id) in entry:
{entry}")
62         mid = str(mid).strip()
63         sp = str(entry.get("split", "")).strip().lower()
64         if sp not in ("train", "eval"):
65             raise ValueError(f"split label must be exactly 'train' or 'eval', got {sp!r}
in entry: {entry}")
66         if sp == "eval":
67             eval_ids.add(mid)
68         else:
69             train_ids.add(mid)
70
71     return train_ids, eval_ids
72
73
74 def assert_disjoint(train_ids: Iterable[str], eval_ids: Iterable[str]) -> None:
75     """Raise if any match-id overlap between train and eval sets.
76
77     This would have caught train/eval leakage (e.g. a video used for both
78     training a segmenter and for regression/eval baselines).
79     """
80     t = set(str(x) for x in train_ids)
81     e = set(str(x) for x in eval_ids)
82     overlap = t & e
83     if overlap:

```

```

84         raise ValueError(
85             f"Train/eval leakage detected on match ids: {sorted(overlap)}. "
86             "This would have caught train/eval leakage."
87         )
88

```

15. user (2026-06-16T02:25:12.143Z)

```

1     """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3     A *systematic, repeatable* version of the one-off ablation
4     (`scratch/gate_ab_ablation.py`):
5     run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
6     per
7     signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
8     bootstrap
9     95% CI + paired sign test + the C4 over-segmentation ratio. See
10    `scratch/ABLATION_FRAMEWORK_PLAN.md`
11    (the design) and `docs/ABLATION_LAYER.md` (the shipped note).
12
13    **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
14    `experiment.compare` + `eval_stats` engine, exactly as `compare` was a driver over
15    `metrics`/`calibration`. For each signal `s` the marginal contribution is:
16
17    ?(s) = score(full set S) - score(S \ {s})          # leave-one-out drop, ?F1
18
19    primary
20
21    operationally `experiment.compare(videos, variant_a=without_s, variant_b=full)` ? A is
22    the
23    ablated (smaller) variant, B is full, so the reported B-A delta IS ?(s), a per-video-
24    paired ?F1
25    with bootstrap CI + exact sign test, LOVO-honest for learned variants.
26
27    Two ablation granularities fall out of one inventory:
28
29    - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
30    - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
31    owner's
32    "is this old SIGNAL still relevant?" question at cue granularity).
33
34    A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
35    weights) or
36    a **gate knob** (Gate-A's `min_crossings` threshold ? ablated by zeroing the knob, NOT
37    by
38    dropping a column). The runner handles both uniformly: `without_s` subtracts the
39    columns AND/OR
40    zeroes the gate knob from `cfg.full`.
41
42    Honest small-N contract (inherited from `eval_stats` / `per_user_gate`; §6 of the
43    plan):
44
45    - never a bare mean ? every row ships a CI + the exact sign test;
46    - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
47    sign test
48    agreeing** (the `per_user_gate` bar, in reverse);
49    - **structural signals are exempt from auto-demotion** ? a guard whose value is in
50    failure modes
51    the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked `structural` and
52    is NEVER
53    flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
54    ? honestly
55    but tags `structural_protected`;
56
57    - the runner emits `earns_keep` / `suggestive` / `inert` /
58    `structural_protected` ? it
59
60    MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
61    proof).

```

```

40
41     Pure + offline + GPU-free. Reuses the persisted golden feature substrate
42     (``fusion_golden.py``
43     writes one ``<name>_golden_features.json`` per video). CLI::
44
45     python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
46     """
47     from __future__ import annotations
48
49     import argparse
50     import glob
51     import json
52     import logging
53     import os
54     from dataclasses import dataclass, field
55     from typing import Any, Dict, List, Optional, Sequence, Tuple
56
57     from backend.eval import experiment
58     from backend.eval.experiment import Variant, VideoCandidates
59
60     logger = logging.getLogger(__name__)
61
62     # --- The signal inventory (the things the runner ablates), grouped by producer.
63     -----
64     # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
65     # (person), audio_features (audio). The shuttle group's velocity block + duration are
66     the
67     # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
68     knob.
69     SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
70     "active_ratio", "net_crossings")
71     PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
72     "mean_player_motion",
73     "in_court_ratio", "shuttle_player_colocation")
74     AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
75     "audio_hit_strength_mean")
76
77     # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
78     EARNNS_KEEP = "earns_keep" # CI clears 0 AND sign test agrees ? removing it
79     hurts
80     SUGGESTIVE = "suggestive" # point estimate leans positive but CI spans 0
81     INERT = "inert" # wash this run (?0, CI spans 0) ? candidate,
82     not proof
83     STRUCTURAL_PROTECTED = "structural_protected" # a guard; never auto-demoted on "no
84     lift"
85
86     #: Human-readable one-liners for each verdict (used in the PDF + table legend).
87     VERDICT_LABEL = {
88         EARNNS_KEEP: "clears-bar",
89         SUGGESTIVE: "suggestive",
90         INERT: "inert",
91         STRUCTURAL_PROTECTED: "structural-protected",
92     }
93
94     @dataclass(frozen=True)
95     class Signal:
96         """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
97
98         - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
99         - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
100         whole group.
101         - ``columns`` ? the persisted feature columns this signal contributes (may be empty
102         for a
103         pure gate signal like Gate-A whose column is shared with the shuttle group).

```

```

94         - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
zeroing it on
95         the full variant rather than dropping a column.
96         - ``structural`` ? a guard whose value is in failure modes the golden corpus may
lack; NEVER
97         flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
``structural=True``).
98         ""
99
100        name: str
101        group: str
102        columns: Tuple[str, ...] = ()
103        gate_knob: Optional[str] = None
104        structural: bool = False
105
106
107    @dataclass
108    class AblationConfig:
109        """How to run an ablation over a corpus.
110
111        - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
112        the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
carries
113        ``feature_names`` = all columns and the gates ON.
114        - ``signals`` ? the inventory to ablate.
115        - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
(LOGO).
116        - ``iou`` ? temporal-IoU match threshold.
117        - ``honest`` ? LOVO for learned variants (the number to trust).
118        - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
119        - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
120        ""
121
122        full: Variant
123        signals: List[Signal]
124        granularity: str = "group"
125        iou: float = 0.5
126        honest: bool = True
127        seed: int = 0
128        p_threshold: float = 0.05
129
130
131    @dataclass
132    class AblationRow:
133        """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a
verdict)."""
134
135        signal: str
136        group: str
137        structural: bool
138        dropped_columns: List[str]
139        gate_knob: Optional[str]
140        n: int
141        delta_f1: float
142        ci_low: float
143        ci_high: float
144        ci_excludes_zero: bool
145        sign_wins: int
146        sign_losses: int
147        sign_p_value: float
148        f1_full: float
149        f1_ablated: float
150        seg_ratio_full: float
151        seg_ratio_ablated: float

```

```

152     verdict: str
153     structural_protected: bool
154
155     def as_dict(self) -> Dict[str, Any]:
156         return {
157             "signal": self.signal,
158             "group": self.group,
159             "structural": self.structural,
160             "dropped_columns": list(self.dropped_columns),
161             "gate_knob": self.gate_knob,
162             "n": self.n,
163             "delta_f1": self.delta_f1,
164             "ci_low": self.ci_low,
165             "ci_high": self.ci_high,
166             "ci_excludes_zero": self.ci_excludes_zero,
167             "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
168                           "p_value": self.sign_p_value},
169             "f1_full": self.f1_full,
170             "f1_ablated": self.f1_ablated,
171             "segment_count_ratio_full": self.seg_ratio_full,
172             "segment_count_ratio_ablated": self.seg_ratio_ablated,
173             "verdict": self.verdict,
174             "structural_protected": self.structural_protected,
175         }
176
177
178     @dataclass
179     class AblationReport:
180         """All signals' rows + the run's provenance ? a structured, JSON-able ablation
181         result."""
182         granularity: str
183         iou: float
184         num_videos: int
185         honest_lovo: bool
186         full_set_signals: List[str]
187         full_spec_hash: str
188         label_set_hash: str
189         rows: List[AblationRow] = field(default_factory=list)
190
191         def sorted_rows(self) -> List[AblationRow]:
192             """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
193             order)."""
194             return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)
195
196         def as_dict(self) -> Dict[str, Any]:
197             return {
198                 "granularity": self.granularity,
199                 "iou": self.iou,
200                 "num_videos": self.num_videos,
201                 "honest_lovo": self.honest_lovo,
202                 "full_set_signals": list(self.full_set_signals),
203                 "full_spec_hash": self.full_spec_hash,
204                 "label_set_hash": self.label_set_hash,
205                 "rows": [r.as_dict() for r in self.sorted_rows()],
206             }
207
208     # ----- the driver
209
210
211     def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
212     Variant:
213         """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
214         its

```

```

213     ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
214     mutates ``full``. """
215     cols = set(columns)
216     fn = full.feature_names
217     ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
218     overrides: Dict[str, Any] = {"feature_names": ablated_fn}
219     if gate_knob == "min_crossings":
220         overrides["min_crossings"] = 0
221     elif gate_knob == "min_players":
222         overrides["min_players"] = 0
223     elif gate_knob is not None:
224         raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
225     dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
226     return experiment.Variant.from_dict(**full.to_dict(), **overrides,
227                                       "name": f"?{dropped_label}")
228
229
230 def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
231              sign_p: float, p_threshold: float) -> Tuple[str, bool]:
232     """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
233
234     A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
235     signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
236     removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
237     wash this run is ``inert`` (a candidate, never "proven useless" at n=6). """
238     if structural:
239         return STRUCTURAL_PROTECTED, True
240     sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
241     if ci_excludes_zero and sign_agrees:
242         return EARNS_KEEP, False
243     if sign_wins > sign_losses:          # leans toward "removal hurts" but evidence is
thin
244         return SUGGESTIVE, False
245     return INERT, False
246
247
248 def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
249              signal: Signal) -> AblationRow:
250     """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
251
252     Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
253     variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
?(s).
254     Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
255     come for free. """
256     ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257     res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                             honest_stats=True)
259     # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
260     # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
261     from backend.eval.eval_stats import paired_delta_ci
262     by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
263     by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
264     vids = [p["video"] for p in res["variant_a"]["per_video"]]
265     pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)

```

```

266     st = pd["sign_test"]
267     sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
268     sign_p = float(st["p_value"])
269     verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
270                                   sign_wins, sign_losses, sign_p, cfg.p_threshold)
271
272     seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
273     seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
274
275     return AblationRow(
276         signal=signal.name,
277         group=signal.group,
278         structural=signal.structural,
279         dropped_columns=list(signal.columns),
280         gate_knob=signal.gate_knob,
281         n=int(pd["n"]),
282         delta_f1=float(pd["mean_delta"]),
283         ci_low=float(pd["ci_low"]),
284         ci_high=float(pd["ci_high"]),
285         ci_excludes_zero=bool(pd["ci_excludes_zero"]),
286         sign_wins=sign_wins,
287         sign_losses=sign_losses,
288         sign_p_value=sign_p,
289         f1_full=float(res["variant_b"]["aggregate"]["f1"]),
290         f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
291         seg_ratio_full=float(seg_full),
292         seg_ratio_ablated=float(seg_abl),
293         verdict=verdict,
294         structural_protected=protected,
295     )
296
297
298     def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
299         """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
300         columns,
301         OR the gate knob, OR the structural flag. First-seen group order is preserved."""
302         order: List[str] = []
303         by_group: Dict[str, List[Signal]] = {}
304         for s in signals:
305             if s.group not in by_group:
306                 order.append(s.group)
307                 by_group[s.group] = []
308             by_group[s.group].append(s)
309         out: List[Signal] = []
310         for g in order:
311             members = by_group[g]
312             cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
313                                                         m.columns))
314             knob = next((m.gate_knob for m in members if m.gate_knob), None)
315             structural = any(m.structural for m in members)
316             out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
317                               structural=structural))
318         return out
319
320
321     def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
322     AblationReport:
323         """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
324         table).
325
326         A pure driver: each row is one ``experiment.compare(without_s, full)``.
327         ``granularity="group"``
328         (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
329         set at a
330         time. The full reference is scored once implicitly per row (cheap re-scoring;

```

```

detection is
324         already persisted)."""
325         if len(videos) < 1:
326             raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
327         if cfg.granularity not in ("signal", "group"):
328             raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")
329
330         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
331         rows = [ablate_one(videos, cfg, s) for s in inventory]
332
333         # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
334         full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
335         from backend.eval.regression import gt_hash
336         all_gts = [iv for vc in videos for iv in (vc.gts or [])]
337         return AblationReport(
338             granularity=cfg.granularity,
339             iou=cfg.iou,
340             num_videos=len(videos),
341             honest_lovo=bool(full_eval["honest_lovo"]),
342             full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
343             full_spec_hash=cfg.full.spec_hash(),
344             label_set_hash=gt_hash(all_gts),
345             rows=rows,
346         )
347
348
349     # ----- default
inventories
350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359         return [
360             Signal("shuttle.duration", "shuttle", ("duration",)),
361             Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362             Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363             Signal("person", "person", PERSON_COLUMNS),
364             Signal("audio", "audio", AUDIO_COLUMNS),
365         ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)

```



```

373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ----- corpus
I/O
384
385
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals +
shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs
predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
390         mirroring ``experiment._feature_column``'s tolerance."""
391         videos: List[VideoCandidates] = []
392         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
393             with open(path, encoding="utf-8") as f:
394                 d = json.load(f)
395                 cands = d["candidates"]
396                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
397                 features: Dict[str, List[float]] = {}
398                 for fn in SHUTTLE_COLUMNS:
399                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
400                 for fn in PERSON_COLUMNS:
401                     features[fn] = [float(c["person"][fn]) for c in cands]
402                 for fn in AUDIO_COLUMNS:
403                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
404                 gts = [(float(a), float(b)) for a, b in d["gts"]]
405                 videos.append(VideoCandidates(name=d["name"], intervals=intervals,
406                                             features=features, gts=gts))
407         return videos
408
409
410     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
411         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
412         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
413         (distinct from the static Gate-A litmus)."""
414         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
415         return experiment.Variant(name="full_learned", feature_names=cols,
416                                 threshold=threshold, min_crossings=min_crossings)
417
418
419     # ----- table
rendering
420
421
422     def format_table(report: AblationReport) -> str:
423         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
424         rows = report.sorted_rows()
425         header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
426                 f" · n={report.num_videos} · IoU {report.iou}")

```

```

427         f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
428         f" . {report.granularity.upper()}")
429     cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
430     widths = [18, 17, 18, 14, 5, 22]
431
432     def _fmt_row(cells: Sequence[str]) -> str:
433         return " ".join(c.ljust(w) for c, w in zip(cells, widths))
434
435     lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
436     for r in rows:
437         ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
438         sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
439         verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
440         lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
441                               "yes" if r.ci_excludes_zero else "no", verdict]))
442     return "\n".join(lines)
443
444
445     # ----- PDF
446 export
447
448     def export_pdf(report: AblationReport, path: str) -> str:
449         """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
450         marginal
451         contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
452         structural-protected). Reuses the muted booktabs aesthetic of
453         ``scratch/quality_report.py``.
454
455         Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
456         GPU)."""
457         from reportlab.lib import colors
458         from reportlab.lib.enums import TA_CENTER, TA_LEFT
459         from reportlab.lib.pagesizes import A4
460         from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
461         from reportlab.lib.units import cm
462         from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table,
463         TableStyle)
464
465         navy = colors.HexColor("#1F3A5F")
466         black = colors.HexColor("#000000")
467         darkgrey = colors.HexColor("#333333")
468         light = colors.HexColor("#F0F0F0")
469         hairline = colors.HexColor("#BFBFBF")
470         row_alt = colors.HexColor("#F7F7F7")
471
472         base = getSampleStyleSheet()["Normal"]
473         title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold",
474                                fontSize=16,
475                                leading=20, alignment=TA_CENTER, textColor=navy,
476                                spaceAfter=4)
477         subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique",
478                                   fontSize=9.5,
479                                   leading=13, alignment=TA_CENTER, textColor=darkgrey,
480                                   spaceAfter=10)
481         h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11,
482                             leading=15,
483                             textColor=navy, spaceBefore=10, spaceAfter=4)
484         body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9,
485                               leading=12.5,
486                               alignment=TA_LEFT, textColor=black, spaceAfter=5)
487         cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2,
488                               leading=10.5,
489                               textColor=black)
490         cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold",

```

```

fontSize=8.2,
480             leading=10.5, textColor=black)
481     head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
482             leading=10.5, textColor=black)
483
484     rows = report.sorted_rows()
485     story: List[Any] = [
486         Paragraph("Signal Ablation & Feature-Enablement Report", title),
487         Paragraph(
488             f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out
marginal "
489             f"contribution &mdash; n={report.num_videos} golden matches, IoU
{report.iou}, "
490             f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
491             "Honest measured results only.",
492             subtitle),
493     ]
494
495     full_set = ", ".join(report.full_set_signals)
496     story.append(Paragraph("Run provenance", h2))
497     story.append(Paragraph(
498         f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
499         f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
500         f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out
"
501         "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with
a "
502         "bootstrap 95% CI and an exact paired sign test &mdash; a signal earns its keep
only "
503         "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test
agreeing.",
504         body))
505
506     story.append(Paragraph("Marginal contribution per signal", h2))
507     table_data: List[List[Any]] = [[
508         Paragraph("Signal", head), Paragraph("Group", head),
509         Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
510         Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
511         Paragraph("Over-seg", head), Paragraph("Verdict", head),
512     ]]
513     for r in rows:
514         ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
515         sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
516         overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
517         verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
518         df1_style = cell_b if r.verdict == EARNs_KEEP else cell
519         table_data.append([
520             Paragraph(r.signal, cell), Paragraph(r.group, cell),
521             Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
522             Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no",
cell),
523             Paragraph(overseg, cell), Paragraph(verdict, cell),
524         ])
525     col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm,
2.6 * cm]
526     tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
527     last = len(table_data) - 1
528     tbl.setStyle(TableStyle([
529         ("BACKGROUND", (0, 0), (-1, 0), light),
530         ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
531         ("TOPPADDING", (0, 0), (-1, -1), 4),
532         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
533         ("LEFTPADDING", (0, 0), (-1, -1), 4),
534         ("RIGHTPADDING", (0, 0), (-1, -1), 4),
535         ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),

```

```

536         ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
537         ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
538         ("LINEBELOW", (0, last), (-1, last), 1.0, black),
539         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
540     ]))
541     story.append(tbl)
542     story.append(Spacer(1, 0.3 * cm))
543
544     story.append(Paragraph("Verdict key", h2))
545     story.append(Paragraph(
546         "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI
clearing 0 "
547         "and the sign test agreeing (the signal earns its keep). "
548         "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but
the CI "
549         "spans 0 (keep, watch as the corpus grows). "
550         "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement
only after a "
551         "trend across multiple runs, never on one wash at small n. "
552         "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the
golden "
553         "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported
honestly but "
554         "never auto-flagged dead weight on 'no measured lift'.",
555         body))
556
557     doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
558                             topMargin=1.8 * cm, bottomMargin=1.8 * cm,
559                             title="Signal Ablation & Feature-Enablement Report",
560                             author="badminton-highlight-indexer")
561     doc.build(story)
562     logger.info("wrote ablation PDF -> %s", path)
563     return path
564
565
566     # -----
567     CLI
568
569     def main() -> None:
570         ap = argparse.ArgumentParser(
571             description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
572             "over experiment.compare + eval_stats; no new scoring math).")
573         ap.add_argument("--features-dir", default="output",
574             help="dir of *_golden_features.json (the persisted golden
substrate)")
575         ap.add_argument("--granularity", choices=("signal", "group"), default="group",
576             help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
577         ap.add_argument("--iou", type=float, default=0.5)
578         ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
579         ap.add_argument("--litmus", action="store_true",
580             help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
581             "instead of the learned full-set ablation")
582         ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
583         ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
584         args = ap.parse_args()
585
586         videos = load_videos(args.features_dir)
587         if len(videos) < 2:
588             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} ")

```

```

589                 f"(found {len(videos)})")
590
591     if args.litmus:
592         cfg = litmus_config()
593         cfg.iou, cfg.seed = args.iou, args.seed
594     else:
595         cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
596                             granularity=args.granularity, iou=args.iou, seed=args.seed)
597
598     report = run_ablation(videos, cfg)
599     print(format_table(report))
600
601     if args.json:
602         with open(args.json, "w", encoding="utf-8") as f:
603             json.dump(report.as_dict(), f, indent=2)
604             print(f"\nwrote report JSON -> {args.json}")
605     if args.pdf:
606         export_pdf(report, args.pdf)
607         print(f"wrote PDF report -> {args.pdf}")
608
609
610 if __name__ == "__main__":
611     main()
612

```

16. user (2026-06-16T02:25:12.686Z)

```

+ bhi-execpolicy
  feat/highlights-runner
+ feat/local-no-ai-segmenter
  fix/compiler-dedup-overlapping-segments
  fix/gemini-per-video-chunk-dir
  fix/wasb-test-mocks-readability-preflight
  fix/wsl-unreadable-source-preflight
* master
+ perf/wasb-fast-frames
  worktree-agent-a771ff1d85e18f2ed
  remotes/origin/HEAD -> origin/master
  remotes/origin/docs/pmf-field-signals
  remotes/origin/feat/highlights-runner
  remotes/origin/feat/local-no-ai-segmenter
  remotes/origin/fix/compiler-dedup-overlapping-segments
  remotes/origin/fix/gemini-per-video-chunk-dir
  remotes/origin/fix/wasb-test-mocks-readability-preflight
  remotes/origin/fix/wsl-unreadable-source-preflight
  remotes/origin/master
  remotes/origin/perf/wasb-fast-frames
---STATUS---

```

master...origin/master

```

?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md

```

17. assistant (2026-06-16T02:25:16.548Z)

The pull failed with "multiple branches" but I'm on `master` tracking `origin/master` cleanly. This is a read-only mapping task, so I'll proceed on local state. Now let me read the test files and the key dependencies (experiment.py, calibrate_wasb.py, distill_local.py, annotations registry) and grep for CLI entrypoints / how these are invoked.

18. user (2026-06-16T02:25:17.431Z)

```

1     """Tests for the regression runner (GS-2): baseline store, regress(), the
2     provider-driven end-to-end path, and the CLI exit-code contract."""
3     import pytest

```

```

4
5 from backend.infrastructure.database import Database
6 from backend.eval import regression
7 import run_regression
8
9
10 @pytest.fixture
11 def db(tmp_path):
12     d = Database(str(tmp_path / "t.db"))
13     d.add_video("vid1", "/x/m.mp4", "processed", [])
14     # Two confirmed segments matching the GT below ? perfect baseline.
15     d.add_play_segment("vid1", 0.0, 10.0)
16     d.add_play_segment("vid1", 20.0, 30.0)
17     return d
18
19
20 GT = [(0.0, 10.0), (20.0, 30.0)]
21
22
23 # ----- baseline store -----
24
25 def test_save_and_load_baseline_roundtrip(tmp_path):
26     p = str(tmp_path / "b.json")
27     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
28     b = regression.load_baselines(p)
29     assert b["vid1::final"]["precision"] == 1.0
30
31
32 def test_load_baselines_absent_is_empty(tmp_path):
33     assert regression.load_baselines(str(tmp_path / "nope.json")) == {}
34
35
36 # ----- regress() -----
37
38 def test_no_baseline_is_not_a_regression(db, tmp_path):
39     r = regression.regress(db, "vid1", GT, baseline_path=str(tmp_path / "b.json"))
40     assert r.regressed is False
41     assert r.precision == 1.0 and r.recall == 1.0
42     assert any("no baseline" in why for why in r.reasons)
43
44
45 def test_matching_baseline_passes(db, tmp_path):
46     p = str(tmp_path / "b.json")
47     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
48     r = regression.regress(db, "vid1", GT, baseline_path=p)
49     assert r.regressed is False
50
51
52 def test_recall_drop_beyond_tolerance_regresses(tmp_path):
53     # DB has only ONE of the two GT rallies ? recall 0.5 vs baseline 1.0.
54     d = Database(str(tmp_path / "t.db"))
55     d.add_video("vid1", "/x/m.mp4", "processed", [])
56     d.add_play_segment("vid1", 0.0, 10.0)
57     p = str(tmp_path / "b.json")
58     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
59
60     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
61     assert r.regressed is True
62     assert r.recall == pytest.approx(0.5)
63     assert any("recall" in why for why in r.reasons)
64
65
66 def test_within_tolerance_does_not_regress(tmp_path):
67     d = Database(str(tmp_path / "t.db"))
68     d.add_video("vid1", "/x/m.mp4", "processed", [])

```

```

69     d.add_play_segment("vid1", 0.0, 10.0)
70     d.add_play_segment("vid1", 20.0, 30.0)
71     p = str(tmp_path / "b.json")
72     # Baseline only 0.02 above current perfect score ? within default 0.05 tolerance.
73     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
74     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
75     assert r.regressed is False
76
77
78 # ----- provider-driven end-to-end (uses the GS-1 FileProvider) -----
79
80 def test_regress_with_provider_file_tier(db, tmp_path):
81     csv = tmp_path / "gt.csv"
82     csv.write_text("start,end\n0,10\n20,30\n")
83     r = regression.regress_with_provider(
84         db, video_id="vid1", video_ref=str(csv), tier="file",
85         baseline_path=str(tmp_path / "b.json"),
86     )
87     assert r.precision == 1.0 and r.recall == 1.0
88
89
90 # ----- CLI exit-code contract (CI gating) -----
91
92 def test_cli_update_baseline_then_clean_run(db, tmp_path, capsys, monkeypatch):
93     # Point the runner at our temp DB via --db; CSV ground truth via 'file' tier.
94     csv = tmp_path / "gt.csv"
95     csv.write_text("start,end\n0,10\n20,30\n")
96     base = str(tmp_path / "b.json")
97     db_path = db.db_path if hasattr(db, "db_path") else str(tmp_path / "t.db")
98
99     # First, snapshot the baseline ? exit 0.
100    rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
101                             "--annotations", str(csv), "--db", db_path,
102                             "--baseline", base, "--update-baseline"])
103    assert rc == 0
104    assert "REGRESSED" not in capsys.readouterr().out
105
106
107 def test_cli_regression_exits_nonzero(tmp_path, capsys):
108     # DB missing one rally vs a baseline of perfect ? regression ? exit 1.
109     d = Database(str(tmp_path / "t.db"))
110     d.add_video("vid1", "/x/m.mp4", "processed", [])
111     d.add_play_segment("vid1", 0.0, 10.0)
112     csv = tmp_path / "gt.csv"
113     csv.write_text("start,end\n0,10\n20,30\n")
114     base = str(tmp_path / "b.json")
115     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
116
117     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
118                             "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
119                             "--baseline", base])
120     assert rc == 1
121     assert "REGRESSED" in capsys.readouterr().out
122
123
124 def test_cli_missing_baseline_exits_3(db, tmp_path):
125     # No baseline + no --update/--allow ? the gate cannot verify ? exit 3 (not a silent
126     # pass).
127     csv = tmp_path / "gt.csv"
128     csv.write_text("start,end\n0,10\n20,30\n")
129     db_path = str(tmp_path / "t.db")
130     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
131                             "--annotations", str(csv), "--db", db_path,
132                             "--baseline", str(tmp_path / "absent.json")])
133     assert rc == 3

```

```

133
134
135 def test_cli_missing_baseline_allowed_exits_0(db, tmp_path):
136     csv = tmp_path / "gt.csv"
137     csv.write_text("start,end\n0,10\n20,30\n")
138     rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
139                             "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
140                             "--baseline", str(tmp_path / "absent.json"),
141                             "--allow-missing-baseline"])
142     assert rc == 0
143
144
145 def test_iou_mismatch_is_incommensurable(db, tmp_path):
146     p = str(tmp_path / "b.json")
147     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
148                             gt_fingerprint=regression.gt_hash(GT))
149     # Same labels but scored at a different IoU ? not a valid comparison.
150     r = regression.regress(db, "vid1", GT, iou_threshold=0.3, baseline_path=p)
151     assert r.regressed is True
152     assert any("incommensurable" in why for why in r.reasons)
153
154
155 def test_changed_labels_detected_via_gt_hash(db, tmp_path):
156     p = str(tmp_path / "b.json")
157     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p, iou_threshold=0.5,
158                             gt_fingerprint=regression.gt_hash(GT))
159     r = regression.regress(db, "vid1", [(0.0, 10.0), (21.0, 31.0)], iou_threshold=0.5,
baseline_path=p)
160     assert r.regressed is True
161     assert any("labels changed" in why for why in r.reasons)
162

```

19. user (2026-06-16T02:25:18.354Z)

```

1     """Canonical GT loader: one reader for BOTH golden-CSV conventions (ADR-008 prereq).
2
3     Pins the frozen golden-label contract: the collector's `curate export` format
4     (`start,end`) and the rally-annotator golden format
5     (`rally_number,start_time,end_time,...`)
6     must load IDENTICALLY through every GT entry point ? the historical two-loader fork
7     (zero/garbage intervals when formats crossed) must never reopen.
8     """
9
10    import csv
11
12    import pytest
13
14    from backend.eval.gt_loader import load_gt_intervals
15    from backend.eval.annotations import load_annotations
16    from backend.eval.calibrate_wasb import load_golden_gts
17
18    EXPECTED = [(5.654, 8.425), (12.928, 18.673), (31.171, 39.173)]
19
20    def _collector_export(path):
21        """Replicates EXACTLY what the collector's export_eval_set writes (curate.py)."""
22        with open(path, "w", newline="", encoding="utf-8") as f:
23            w = csv.writer(f)
24            w.writerow(["start", "end"])
25            for s, e in EXPECTED:
26                w.writerow([s, e])
27        return str(path)
28
29    def _annotator_golden(path):
30        """Replicates the rally-annotator .rallies.csv schema."""

```



```

31         with open(path, "w", newline="", encoding="utf-8") as f:
32             w = csv.writer(f)
33             w.writerow(["rally_number", "start_time", "end_time", "ending_reason", "sport"])
34             for i, (s, e) in enumerate(EXPECTED, 1):
35                 w.writerow([i, s, e, "winner", "badminton"])
36         return str(path)
37
38
39     @pytest.mark.parametrize("loader", [load_gt_intervals, load_annotations,
load_golden_gts])
40     def test_collector_export_format_loads_everywhere(tmp_path, loader):
41         p = _collector_export(tmp_path / "export.csv")
42         assert loader(p) == EXPECTED
43
44
45     @pytest.mark.parametrize("loader", [load_gt_intervals, load_annotations,
load_golden_gts])
46     def test_annotator_golden_format_loads_everywhere(tmp_path, loader):
47         p = _annotator_golden(tmp_path / "golden.csv")
48         assert loader(p) == EXPECTED
49
50
51     def test_round_trip_export_equals_golden(tmp_path):
52         """The fork regression: both conventions of the SAME rallies -> identical
intervals."""
53         a = load_gt_intervals(_collector_export(tmp_path / "a.csv"))
54         b = load_gt_intervals(_annotator_golden(tmp_path / "b.csv"))
55         assert a == b == EXPECTED
56
57
58     def test_headerless_positional_and_mmss(tmp_path):
59         p = tmp_path / "h.csv"
60         p.write_text("# comment\n00:01:12,00:01:39\n102.5,131.0\n", encoding="utf-8")
61         assert load_gt_intervals(str(p)) == [(72.0, 99.0), (102.5, 131.0)]
62
63
64     def test_strict_raises_with_location(tmp_path):
65         p = tmp_path / "bad.csv"
66         p.write_text("start,end\n10,5\n", encoding="utf-8") # reversed
67         with pytest.raises(ValueError, match="bad.csv:2"):
68             load_gt_intervals(str(p), strict=True)
69         p2 = tmp_path / "neg.csv"
70         p2.write_text("start,end\n-1,5\n", encoding="utf-8")
71         with pytest.raises(ValueError, match="negative"):
72             load_gt_intervals(str(p2), strict=True)
73
74
75     def test_lenient_skips_and_logs(tmp_path, caplog):
76         p = tmp_path / "mixed.csv"
77
78         p.write_text("rally_number,start_time,end_time\n1,1.0,2.0\n2,,\n3,9.0,4.0\n4,5.0,6.0\n",
79                     encoding="utf-8")
79         with caplog.at_level("WARNING"):
80             out = load_gt_intervals(str(p), strict=False)
81             assert out == [(1.0, 2.0), (5.0, 6.0)]
82             assert "skipped 2" in caplog.text
83
84
85     def test_overlap_warning(tmp_path, caplog):
86         p = tmp_path / "ov.csv"
87         p.write_text("start,end\n1,10\n5,12\n", encoding="utf-8")
88         with caplog.at_level("WARNING"):
89             out = load_gt_intervals(str(p))
90             assert len(out) == 2
91             assert "overlapping" in caplog.text.lower()

```

20. user (2026-06-16T02:25:18.839Z)

```

1      """GPU-free tests for the run-telemetry wire-in to the fusion_golden extractor.
2
3      `run()` builds a `RunTelemetry` black box in `out_dir` and emits video_start /
video_done events
4      around each extract_video; a hard kill during a long GPU run thus leaves a forensic
runlog. These
5      tests stub extract_video (no GPU) and inject deterministic collectors + clock via a
patched
6      RunTelemetry factory, so the JSONL content is asserted with no real nvidia-smi / psutil
/ wall-clock.
7      A telemetry-collector failure must NOT break extraction (graceful degradation).
8      """
9      import functools
10     import json
11     import os
12
13     from backend.eval import fusion_golden
14     from backend.utils import run_telemetry as rt
15
16
17     class _Clock:
18         def __init__(self, t=1000.0):
19             self.t = t
20
21         def __call__(self):
22             return self.t
23
24         def tick(self, dt):
25             self.t += dt
26
27
28     def _fake_gpu():
29         return {"temp_c": 70.0, "util_pct": 80.0, "throttled": False}
30
31
32     def _fake_sys():
33         return {"cpu_pct": 50.0, "ram_used_gb": 21.0}
34
35
36     def _patch_telemetry(monkeypatch, gpu_fn=_fake_gpu, sys_fn=_fake_sys):
37         """Make run()'s `RunTelemetry(...)` hermetic: inject deterministic collectors +
clock and a
38         fixed GPU-name header (no real nvidia-smi). Returns nothing; run() builds the
instance itself."""
39         monkeypatch.setattr(rt, "gpu_name", lambda: "FakeGPU")
40         factory = functools.partial(rt.RunTelemetry, gpu_fn=gpu_fn, sys_fn=sys_fn,
clock=_Clock())
41         monkeypatch.setattr(fusion_golden, "RunTelemetry", factory)
42
43
44     def _write_manifest(tmp_path, names):
45         manifest = [
46             {"name": n, "trajectory": f"{n}.csv", "fps": 30.0, "frame_width": 1920.0,
47              "labels": f"{n}.rallies.csv"}
48             for n in names
49         ]
50         mp = str(tmp_path / "m.json")
51         with open(mp, "w", encoding="utf-8") as f:
52             json.dump(manifest, f)
53         return mp
54

```

```

55
56     def _read_lines(path):
57         with open(path, encoding="utf-8") as f:
58             return [ln for ln in f.read().splitlines() if ln]
59
60
61     def test_run_writes_telemetry_with_start_and_done_events(tmp_path, monkeypatch):
62         """run() drops run_telemetry.jsonl in out_dir: a meta header +
video_start/video_done lines."""
63         _patch_telemetry(monkeypatch)
64         out_dir = str(tmp_path / "out")
65         os.makedirs(out_dir)
66
67         def _stub(spec, **kw):
68             # A real telemetry instance is threaded through; drive one snapshot from inside
extraction.
69             telem = kw.get("telemetry")
70             if telem is not None:
71                 telem.snapshot(5, 10, phase=spec["name"])
72             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
73                     "candidates": [{"label": 1}, {"label": 0}], "gts": []}
74
75         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
76
77         fusion_golden.run(_write_manifest(tmp_path, ["alpha", "beta"]), out_dir)
78
79         rt_path = os.path.join(out_dir, "run_telemetry.jsonl")
80         assert os.path.isfile(rt_path)
81         lines = _read_lines(rt_path)
82         recs = [json.loads(ln) for ln in lines]
83         # meta header is always the first line.
84         assert recs[0]["meta"]["label"] == "fusion_golden"
85         events = [r for r in recs if "event" in r]
86         starts = [r for r in events if r["event"] == "video_start"]
87         done = [r for r in events if r["event"] == "video_done"]
88         assert {r["msg"] for r in starts} == {"alpha", "beta"}
89         # video_done carries the per-video window/positive counts for the post-mortem.
90         done_by = {r["msg"]: r for r in done}
91         assert done_by["alpha"]["windows"] == 2 and done_by["alpha"]["positives"] == 1
92         # health is attached to every event (the machine state at each milestone).
93         assert starts[0]["gpu"]["temp_c"] == 70.0 and starts[0]["sys"]["cpu_pct"] == 50.0
94         # every body line is valid JSON.
95         for ln in lines:
96             json.loads(ln)
97
98
99     def test_run_records_error_event_then_reraises(tmp_path, monkeypatch):
100         """A death inside extract_video records an `error` event (with machine state) then
re-raises."""
101         _patch_telemetry(monkeypatch)
102         out_dir = str(tmp_path / "out")
103         os.makedirs(out_dir)
104
105         def _boom(spec, **kw):
106             raise RuntimeError("GPU vanished")
107
108         monkeypatch.setattr(fusion_golden, "extract_video", _boom)
109
110         raised = False
111         try:
112             fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
113         except RuntimeError as e:
114             raised = True
115             assert "GPU vanished" in str(e)
116         assert raised, "extract_video failure must propagate"

```

```

117
118     recs = [json.loads(ln) for ln in _read_lines(os.path.join(out_dir,
"run_telemetry.jsonl"))]
119     errs = [r for r in recs if r.get("event") == "error"]
120     assert len(errs) == 1
121     assert errs[0]["msg"] == "alpha" and "GPU vanished" in errs[0]["err"]
122     assert errs[0]["gpu"]["temp_c"] == 70.0    # machine state captured at the moment of
death
123
124
125     def test_telemetry_collector_failure_does_not_break_run(tmp_path, monkeypatch):
126         """A raising GPU collector degrades gracefully ? extraction still completes and
writes output."""
127         def _exploding_gpu():
128             raise RuntimeError("nvidia-smi exploded")
129
130         _patch_telemetry(monkeypatch, gpu_fn=_exploding_gpu, sys_fn=lambda: None)
131         out_dir = str(tmp_path / "out")
132         os.makedirs(out_dir)
133
134         def _stub(spec, **kw):
135             telem = kw.get("telemetry")
136             if telem is not None:
137                 telem.snapshot(1, 10, phase=spec["name"])    # must not raise despite the bad
collector
138                 return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
139                     "candidates": [], "gts": []}
140
141         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
142
143         written = fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
144
145         assert written == [os.path.join(out_dir, "alpha_golden_features.json")]
146         assert os.path.isfile(written[0])
147         recs = [json.loads(ln) for ln in _read_lines(os.path.join(out_dir,
"run_telemetry.jsonl"))]
148         # The run survived; events still recorded, just without a gpu/sys block.
149         assert any(r.get("event") == "video_done" for r in recs)
150         assert all("gpu" not in r for r in recs if "event" in r)
151
152
153     def test_run_survives_telemetry_construction_failure(tmp_path, monkeypatch):
154         """If RunTelemetry construction itself raises, run() must still extract (telemetry
is best-effort)."""
155         def _boom_ctor(*a, **k):
156             raise RuntimeError("cannot build telemetry")
157
158         monkeypatch.setattr(fusion_golden, "RunTelemetry", _boom_ctor)
159         out_dir = str(tmp_path / "out")
160         os.makedirs(out_dir)
161
162         def _stub(spec, **kw):
163             assert kw.get("telemetry") is None    # graceful fallback to no telemetry
164             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
165                 "candidates": [], "gts": []}
166
167         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
168
169         written = fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
170         assert written == [os.path.join(out_dir, "alpha_golden_features.json")]
171

```

21. user (2026-06-16T02:25:19.131Z)

```

1         """Tests for the evaluation harness (DB read -> score -> report) and the CLI

```

```

2 glue. Uses a real temporary SQLite DB so the query methods are exercised."""
3
4 import json
5 import pytest
6
7 from backend.infrastructure.database import Database
8 from backend.eval.harness import (
9     evaluate, get_predictions, format_report_text, report_to_dict, _rows_to_intervals,
10 )
11 import run_eval
12
13
14 @pytest.fixture
15 def db(tmp_path):
16     d = Database(str(tmp_path / "test.db"))
17     d.add_video("vid1", "/x/match.mp4", "processed", [])
18     return d
19
20
21 def test_rows_to_intervals_sorts():
22     rows = [{"start_time": 50, "end_time": 60}, {"start_time": 0, "end_time": 10}]
23     assert _rows_to_intervals(rows) == [(0.0, 10.0), (50.0, 60.0)]
24
25
26 def test_get_predictions_final(db):
27     db.add_play_segment("vid1", 0.0, 10.0)
28     db.add_play_segment("vid1", 20.0, 30.0)
29     assert get_predictions(db, "vid1", "final") == [(0.0, 10.0), (20.0, 30.0)]
30
31
32 def test_get_predictions_candidates_with_detector_filter(db):
33     db.add_candidate_segment("vid1", "yolo_hybrid", 0.0, 10.0)
34     db.add_candidate_segment("vid1", "tracknet_hybrid", 40.0, 50.0)
35     yolo = get_predictions(db, "vid1", "candidates", detector="yolo_hybrid")
36     assert yolo == [(0.0, 10.0)]
37     allc = get_predictions(db, "vid1", "candidates")
38     assert len(allc) == 2
39
40
41 def test_get_predictions_unknown_stage_raises(db):
42     with pytest.raises(ValueError):
43         get_predictions(db, "vid1", "bogus")
44
45
46 def test_evaluate_both_stages(db):
47     # GT: two rallies. Candidates catch both; final only confirms one.
48     db.add_candidate_segment("vid1", "yolo_hybrid", 0.0, 10.0)
49     db.add_candidate_segment("vid1", "yolo_hybrid", 20.0, 30.0)
50     db.add_play_segment("vid1", 0.0, 10.0)
51
52     gt = [(0.0, 10.0), (20.0, 30.0)]
53     report = evaluate(db, "vid1", gt, ["candidates", "final"], iou_threshold=0.5)
54
55     cand = report.stages["candidates"].score
56     final = report.stages["final"].score
57     # Candidates find both rallies (detection is fine)...
58     assert cand.recall == 1.0
59     # ...but final only confirms one (segmentation/handoff dropped the other).
60     assert final.recall == pytest.approx(0.5)
61     assert final.true_positives == 1
62
63
64 def test_evaluate_with_pr_curve(db):
65     db.add_play_segment("vid1", 0.0, 10.0)
66     report = evaluate(db, "vid1", [(0.0, 10.0)], ["final"], with_pr_curve=True)

```

```

67         assert len(report.stages["final"].pr_curve) == 9
68
69
70     def test_format_report_text_contains_stage_rows(db):
71         db.add_play_segment("vid1", 0.0, 10.0)
72         report = evaluate(db, "vid1", [(0.0, 10.0)], ["final"])
73         text = format_report_text(report)
74         assert "vid1" in text
75         assert "final" in text
76
77
78     def test_format_report_failures_lists_misses(db):
79         # GT has a rally with no matching prediction -> should appear as a miss.
80         report = evaluate(db, "vid1", [(0.0, 10.0)], ["final"])
81         text = format_report_text(report, show_failures=True)
82         assert "missed rallies" in text
83         assert "00:00:00" in text # the missed rally's start timestamp
84
85
86     def test_report_to_dict_is_json_serialisable(db):
87         db.add_play_segment("vid1", 0.0, 10.0)
88         report = evaluate(db, "vid1", [(0.0, 10.0)], ["final"], with_pr_curve=True)
89         d = report_to_dict(report)
90         json.dumps(d) # must not raise
91         assert d["video_id"] == "vid1"
92         assert d["stages"]["final"]["score"]["f1"] == 1.0
93
94
95     # ----- CLI glue
96
97     def test_cli_scaffold(tmp_path, capsys):
98         ann = str(tmp_path / "rallies.csv")
99         rc = run_eval.main(["--scaffold", "--annotations", ann])
100         assert rc == 0
101         from backend.eval.annotations import load_annotations
102         assert load_annotations(ann) == [] # header-only template
103
104
105     def test_cli_requires_annotations(capsys):
106         assert run_eval.main([]) == 2
107
108
109     def test_cli_missing_db_errors(tmp_path):
110         ann = str(tmp_path / "r.csv")
111         from backend.eval.annotations import write_scaffold
112         write_scaffold(ann)
113         rc = run_eval.main(["--video-id", "abc", "--annotations", ann,
114                             "--db", str(tmp_path / "nope.db")])
115         assert rc == 2
116
117
118     def test_cli_full_run(tmp_path, capsys):
119         db_path = str(tmp_path / "sports.db")
120         d = Database(db_path)
121         d.add_video("vid1", "/x/m.mp4", "processed", [])
122         d.add_play_segment("vid1", 0.0, 10.0)
123
124         ann = tmp_path / "r.csv"
125         ann.write_text("start,end\n0,10\n")
126
127         json_out = str(tmp_path / "out.json")
128         rc = run_eval.main(["--video-id", "vid1", "--annotations", str(ann),
129                             "--db", db_path, "--stage", "final", "--json", json_out])
130         assert rc == 0
131         captured = capsys.readouterr().out

```

```

132         assert "Rally evaluation" in captured
133         with open(json_out) as f:
134             report = json.load(f)
135         assert report["stages"]["final"]["score"]["f1"] == 1.0
136
137
138     def test_get_file_md5_stable(tmp_path):
139         f = tmp_path / "v.mp4"
140         f.write_bytes(b"hello world")
141         h1 = run_eval.get_file_md5(str(f))
142         h2 = run_eval.get_file_md5(str(f))
143         assert h1 == h2 and len(h1) == 32
144

```

22. assistant (2026-06-16T02:25:22.423Z)

Now let me read the remaining two tests, the key dependencies (experiment.py, calibrate_wasb.py, distill_local.py), and the CLI entrypoints. Let me also locate any committed golden fixtures and manifest files.

23. user (2026-06-16T02:25:23.402Z)

```

1     """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3     The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
4     `min_crossings=2`) is **mock-tested only** on real video (`test_fusion_hybrid.py`
drives a
5     MagicMock runner). The offline ablation litmus
6     (`test_ablation.test_litmus_reproduces_gate_a`)
7     validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8     (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
9     runs the real `FusionSegmenter` decision seams (`enrich_candidate_stats` net-
crossings +
10    `_select_for_handoff` Gate A) over windows from the **production**
11    `trajectory_to_action_windows` config, on the 6 cached golden WASB trajectories, and
asserts
12    the served path's measured behaviour.
13
14    GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
15    CI where the golden trajectories are absent; runs on the owner box where they are
present.
16
17    HONEST FINDING baked into the assertions (2026-06-14): on the **production windowing**
Gate A is
18    NOT the +0.074 win the offline harness reports. Production windowing is already
conservative
19    (velocity_threshold 0.02, merge_gap 2.0, min_window 2.0) ? few long windows, over-seg
ratio
20    already < 1 ? there is little held-shuttle junk left for Gate A to cut. So the served
?F1 is
21    ~neutral (slightly negative), NOT +0.074. The +0.074 reproduces ONLY under the
permissive
22    proposal windowing (asserted in `test_proposal_windowing_reproduces_offline_lift`
below ? that
23    is what the offline litmus measures). These tests pin BOTH facts so a future windowing
change
24    that silently revives ? or breaks ? either one trips the litmus.
25    """
26    from __future__ import annotations
27
28    import json
29    import os

```

```

29
30     import pytest
31
32     # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
33     # (CI), skip the whole module rather than erroring at import.
34     pytest.importorskip("cv2")
35
36     from backend.eval import served_gate_a as sga # noqa: E402
37     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
38     from backend.pipeline.detectors import rally_gate # noqa: E402
39     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
40     from backend.eval.metrics import score # noqa: E402
41     from backend.eval import segmentation_metrics as _segm # noqa: E402
42
43     # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
44     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
45     # out-of-tree checkout.
46     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
47     MANIFEST = os.environ.get(
48         "SERVED_GATE_A_MANIFEST",
49         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
50     )
51
52
53     def _golden_specs():
54         """Return the manifest specs whose trajectory + label files are all present, else
[ ]."""
55         if not os.path.isfile(MANIFEST):
56             return []
57         with open(MANIFEST, encoding="utf-8") as f:
58             specs = json.load(f)
59         ok = [s for s in specs
60              if os.path.isfile(str(s.get("trajectory", "")))
61              and os.path.isfile(str(s.get("labels", "")))]
62         return ok if len(ok) >= 6 else []
63
64
65     _SPECS = _golden_specs()
66     _skip = pytest.mark.skipif(
67         not _SPECS,
68         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
69     )
70
71
72     # ----- served-path
litmus
73
74     @_skip
75     def test_served_path_runs_on_all_six_golden_videos():
76         """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
all 6
77         golden trajectories and yields a handoff window list for each, for both gate
settings."""
78         results = sga.evaluate_manifest(_SPECS)
79         assert len(results) == 6
80         for r in results:
81             assert r.num_gt > 0
82             assert r.n_handoff_off >= r.n_handoff_on # Gate A only ever drops windows,
never adds
83
84
85     @_skip

```



```

86     def test_served_gate_a_does_not_regress_f1():
87         """SERVED no-regression guard: on the production windowing Gate A must not
MATERIALLY hurt
88         F1. It is ~neutral here (the held-shuttle junk the offline harness sees is already
merged
89         away by the conservative production windowing) ? so we assert the mean ?F1 sits in a
tight
90         band around 0, never a real regression. (The +0.074 lift lives under the permissive
proposal
91         windowing ? see the dedicated test below.)"""
92         results = sga.evaluate_manifest(_SPECS)
93         agg = sga.aggregate(results)
94         # Honest measured value (2026-06-14): mean served ?F1 ? -0.008. Pin a tolerance band
so a
95         # real regression (Gate A actively destroying served F1) trips, while the measured
~neutral
96         # result passes.
97         assert -0.03 <= agg["mean_delta_f1"] <= 0.03, agg
98         # And Gate A must never collapse served F1 to ~0 on the production path.
99         assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
100
101
102     @skip
103     def test_served_gate_a_does_not_increase_over_segmentation():
104         """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when
it's on
105         (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
106         results = sga.evaluate_manifest(_SPECS)
107         agg = sga.aggregate(results)
108         assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg
109         # Per-video too: the gate is monotone (kept windows ? all windows).
110         for r in results:
111             assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r
112
113
114     @skip
115     def test_served_uses_production_windowing_operating_point():
116         """Pin the served operating point to config.json's indexing defaults ? if someone
retunes
117         production windowing, this litmus's premise (and the divergence it documents) is
revisited
118         deliberately, not silently."""
119         assert sga.PROD_VELOCITY_THRESH == 0.02
120         assert sga.PROD_MERGE_GAP == 2.0
121         assert sga.PROD_MIN_WINDOW == 2.0
122
123
124     # ----- the +0.074 lives under PROPOSAL
windowing
125
126     # distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the
OFFLINE
127     # ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-dur ?
many
128     # small windows). The +0.074 Gate-A lift is a property of THIS windowing, not the served
one.
129     _PROPOSAL = (0.005, 1.0, 0.5)
130
131
132     @skip
133     def test_proposal_windowing_reproduces_offline_lift():
134         """CONTROL / root-cause pin: under the OFFLINE harness's permissive proposal
windowing, the
135         SAME ``rally_gate`` net-crossing gate the served seam uses reproduces the offline
litmus

```

```

136         (?F1 ? +0.074, 6/6 wins, over-seg 1.68?1.01). This proves the served ~neutral result
is a
137 WINDOWING difference, not a broken gate ? and locks the offline number to its true
premise. """
138     d_on, d_off, seg_on, seg_off = [], [], [], []
139     for s in _SPECS:
140         fps, fw = float(s["fps"]), float(s["frame_width"])
141         pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
142         wins = TrackNetRunner.trajectory_to_action_windows(
143             pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
144             merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
145         iv = [(w["start"], w["end"]) for w in wins]
146         est = rally_gate.estimate_play_axis(pts)
147         gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
148         on = [(g.start, g.end) for g in gated if g.crossings >= 2]
149         gts = load_golden_gts(s["labels"])
150         d_on.append(score(on, gts).f1)
151         d_off.append(score(iv, gts).f1)
152         seg_on.append(_segm.segment_count_ratio(on, gts))
153         seg_off.append(_segm.segment_count_ratio(iv, gts))
154     n = len(d_on)
155     mean_delta = (sum(d_on) - sum(d_off)) / n
156     wins = sum(1 for a, b in zip(d_on, d_off) if a > b + 1e-9)
157     assert mean_delta == pytest.approx(0.074, abs=0.01), mean_delta
158     assert wins == 6, wins
159     assert sum(seg_off) / n == pytest.approx(1.68, abs=0.05)
160     assert sum(seg_on) / n == pytest.approx(1.01, abs=0.05)
161

```

24. user (2026-06-16T02:25:23.855Z)

```

1     """Tuned-segment export for eyeball verification (pure parts; no WSL/GPU)."""
2     import csv
3
4     from backend.eval import export_wasb_segments as ex
5     from backend.tools.rally_slicer import load_rally_labels
6
7
8     def _write_traj(path):
9         """Synthetic trajectory: two moving segments separated by an invisible gap."""
10        rows = [("Frame", "Visibility", "X", "Y")]
11        for i in range(0, 30):
12            rows.append((i, 1, i * 5, 50))
13        for i in range(30, 50):
14            rows.append((i, 0, -1, -1))
15        for i in range(50, 80):
16            rows.append((i, 1, (i - 50) * 5, 50))
17        with open(path, "w", newline="") as f:
18            csv.writer(f).writerows(rows)
19        return str(path)
20
21
22    def _write_golden(path):
23        with open(path, "w", newline="", encoding="utf-8") as f:
24            w = csv.writer(f)
25            w.writerow(["rally_number", "start_time", "end_time"])
26            w.writerow([1, 0.0, 3.0])
27            w.writerow([2, 5.0, 8.0])
28        return str(path)
29
30
31    def test_seconds_to_mmss():
32        assert ex.seconds_to_mmss(106.031) == "1:46.031"
33        assert ex.seconds_to_mmss(6.031) == "0:06.031"
34        assert ex.seconds_to_mmss(-5) == "0:00.000"

```

```

35
36
37 def test_segments_to_rows_numbers_in_time_order():
38     rows = ex.segments_to_rows([(1.0, 3.0), (5.5, 8.25)])
39     assert [r["rally_number"] for r in rows] == ["1", "2"]
40     assert rows[0]["start_time"] == "1.000" and rows[0]["end_time"] == "3.000"
41     assert rows[1]["duration"] == "2.750"
42     assert rows[1]["start_mmss"] == "0:05.500"
43
44
45 def test_exported_csv_feeds_straight_into_the_slicer(tmp_path):
46     """The export schema must be exactly what rally_slicer consumes (the reuse
claim)."""
47     out = str(tmp_path / "segs.csv")
48     ex.write_segments_csv(out, [(1.0, 3.0), (5.5, 8.25)])
49     assert load_rally_labels(out) == {"1": (1.0, 3.0), "2": (5.5, 8.25)}
50
51
52 def test_build_comparison_classifies_match_miss_and_false_positive():
53     preds = [(0.0, 3.0), (5.0, 8.0), (20.0, 22.0)] # 3rd has no golden -> false
positive
54     gts = [(0.1, 3.1), (5.0, 8.0), (10.0, 13.0)] # 3rd has no pred -> miss
55     comp = ex.build_comparison(preds, gts, iou_threshold=0.5)
56     sc = comp["score"]
57     assert (sc.true_positives, sc.false_negatives, sc.false_positives) == (2, 1, 1)
58
59     golden = [r for r in comp["rows"] if r["type"] == "golden"]
60     assert len(golden) == 3
61     assert [r["status"] for r in golden].count("matched") == 2
62     assert [r["status"] for r in golden].count("MISS") == 1
63     extra = [r for r in comp["rows"] if r["type"] == "pred_extra"]
64     assert len(extra) == 1 and extra[0]["status"] == "false_positive"
65     assert extra[0]["start_time"] == "20.000"
66
67
68 def test_run_writes_segments_and_comparison(tmp_path):
69     traj = _write_traj(tmp_path / "t.csv")
70     gold = _write_golden(tmp_path / "g.csv")
71     out = str(tmp_path / "out" / "segs.csv")
72     res = ex.run(traj, fps=10.0, frame_width=100.0, out_csv=out,
73                 labels_csv=gold, iou=0.5, baseline=True)
74
75     assert res["num_tuned"] >= 1
76     # tuned CSV exists and is slicer-loadable with the reported number of rallies
77     assert len(load_rally_labels(out)) == res["num_tuned"]
78     # baseline + comparison artifacts written
79     assert load_rally_labels(res["baseline_csv"])
80     assert res["comparison"]["num_gt"] == 2
81     with open(res["comparison_csv"], encoding="utf-8") as f:
82         comp_rows = list(csv.DictReader(f))
83     assert any(r["type"] == "golden" for r in comp_rows)
84
85
86 def test_run_min_crossings_gates_low_crossing_segments(tmp_path):
87     # synthetic segments move monotonically in X -> cross the net <2x -> gate(K=2) drops
them
88     traj = _write_traj(tmp_path / "t.csv")
89     r0 = ex.run(traj, fps=10.0, frame_width=100.0, out_csv=str(tmp_path / "o0.csv"),
min_crossings=0)
90     r2 = ex.run(traj, fps=10.0, frame_width=100.0, out_csv=str(tmp_path / "o2.csv"),
min_crossings=2)
91     assert r2["num_tuned"] < r0["num_tuned"]
92

```

```

run_all_tests.py
run_eval.py
run_regression.py
backend\reporting\run_signals.py
scratch\run_gx010125.py
scratch\run_gx010125_rescore.py
scratch\run_gx010125_full.py
backend\utils\run_telemetry.py
scripts\run_phase3_eval.py
scripts\run_process_and_stitch.py
.claude\worktrees\bhi-execpolicy\backend\reporting\run_signals.py
.claude\worktrees\bhi-execpolicy\backend\utils\run_telemetry.py
.claude\worktrees\bhi-execpolicy\run_all_tests.py
.claude\worktrees\bhi-execpolicy\run_eval.py
.claude\worktrees\bhi-execpolicy\run_regression.py
.claude\worktrees\bhi-execpolicy\scripts\run_phase3_eval.py
.claude\worktrees\bhi-execpolicy\scripts\run_process_and_stitch.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\reporting\run_signals.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\utils\run_telemetry.py
.claude\worktrees\agent-a771ff1d85e18f2ed\run_all_tests.py
.claude\worktrees\agent-a771ff1d85e18f2ed\run_eval.py
.claude\worktrees\agent-a771ff1d85e18f2ed\run_regression.py
.claude\worktrees\agent-a771ff1d85e18f2ed\scripts\run_phase3_eval.py
.claude\worktrees\agent-a771ff1d85e18f2ed\scripts\run_process_and_stitch.py
.claude\worktrees\local-noai\backend\reporting\run_signals.py
.claude\worktrees\local-noai\backend\utils\run_telemetry.py
.claude\worktrees\local-noai\run_all_tests.py
.claude\worktrees\local-noai\run_eval.py
.claude\worktrees\local-noai\run_regression.py
.claude\worktrees\local-noai\scripts\run_phase3_eval.py
.claude\worktrees\local-noai\scripts\run_process_and_stitch.py

```

26. user (2026-06-16T02:25:24.618Z)

No files found

27. assistant (2026-06-16T02:25:27.606Z)

No `.rallies.csv` committed. Let me read the CLI entrypoints (`run_eval.py`, `run_regression.py`), the experiment engine, the `calibrate_wasb/distill_local` dependencies, and search for committed golden fixtures, manifests, `eval_baselines`, and how these are wired into CI.

28. user (2026-06-16T02:25:28.561Z)

```

1      """CLI: regression-test the production tool against golden-set ground truth.
2
3      Obtains ground truth from an annotation provider (a tier), scores the video's
4      stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5      detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7      Examples
8      -----
9      Snapshot the current numbers as the baseline (first run / after a real improvement):
10         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12         Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14     """
15     import os
16     import sys
17     import json
18     import argparse

```

```

19     import logging
20
21     from backend.infrastructure.database import Database
22     from backend.config import load_config
23     from backend.annotations import annotation_registry
24     from backend.eval import regression
25
26
27     def _default_db_path() -> str:
28         cfg = load_config()
29         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32     def build_parser() -> argparse.ArgumentParser:
33         p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34         p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35         p.add_argument("--tier", default="file",
36                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37         p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38         p.add_argument("--stage", choices=("candidates", "final"), default="final")
39         p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40         p.add_argument("--iou", type=float, default=0.5)
41         p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42         p.add_argument("--db", default=None)
43         p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44         p.add_argument("--update-baseline", action="store_true",
45                         help="Snapshot the current numbers as the new baseline instead of
checking.")
46         p.add_argument("--allow-missing-baseline", action="store_true",
47                         help="Treat a missing baseline as a pass (exit 0) instead of a gate
error (exit 3).")
48         p.add_argument("--json", dest="json_out", default=None)
49         return p
50
51
52     def main(argv=None) -> int:
53         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
54         args = build_parser().parse_args(argv)
55
56         db_path = args.db or _default_db_path()
57         if not os.path.exists(db_path):
58             print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
59             return 2
60         db = Database(db_path)
61
62         result = regression.regress_with_provider(
63             db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
64             stage=args.stage, iou_threshold=args.iou, detector=args.detector,
65             tolerance=args.tolerance, baseline_path=args.baseline,
66         )
67
68         print(regression.format_result(result))
69
70         if args.update_baseline:
71             regression.save_baseline(result.video_id, result.stage, result.precision,
72                                     result.recall, result.f1, path=args.baseline,
73                                     iou_threshold=result.iou_threshold,
74             detector=args.detector,

```

```

74             gt_fingerprint=result.gt_hash)
75         print(f"Baseline updated in {args.baseline}.")
76
77     if args.json_out:
78         with open(args.json_out, "w") as f:
79             json.dump(result.as_dict(), f, indent=2)
80             print(f"JSON written to {args.json_out}")
81
82     # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
83     if result.regressed:
84         return 1
85     if not result.has_baseline and not args.update_baseline and not
args.allow_missing_baseline:
86         print("ERROR: no baseline to compare against ? the gate cannot verify anything.
"
87             "Snapshot one with --update-baseline (then commit it), or pass "
88             "--allow-missing-baseline to accept this run.", file=sys.stderr)
89         return 3
90     return 0
91
92
93 if __name__ == "__main__":
94     raise SystemExit(main())
95

```

29. user (2026-06-16T02:25:29.036Z)

```

1     """CLI to evaluate detected rallies against hand-annotated ground truth.
2
3     Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
4     API calls, fully deterministic. See docs/EVALUATION.md for the workflow.
5
6     Examples
7     -----
8     Create an empty annotation file to fill in:
9         python run_eval.py --scaffold --annotations rallies.csv
10
11     Score a video's detected rallies (identify it by file or by stored video_id):
12         python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
13         python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
14
15     Show which rallies were missed / spurious, and dump a JSON report:
16         python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
17     """
18
19     import os
20     import sys
21     import json
22     import argparse
23     import logging
24
25     from backend.infrastructure.database import Database
26     from backend.config import load_config
27     from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
28     from backend.eval.annotations import load_annotations, write_scaffold
29     from backend.eval.harness import evaluate, format_report_text, report_to_dict, STAGES
30
31
32     def _default_db_path() -> str:
33         """Resolve the DB path from the typed config (output.output_dir /
output.db_name)."""
34         cfg = load_config()
35         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
36

```

```

37
38 _EPILOG = """\
39 Prerequisite:
40 The harness scores predictions read from the SQLite DB ? it does NOT run the
41 pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
42 so its detections land in output/sports_indexer.db. Then point this tool at the
43 same video file (--video) or its stored MD5 (--video-id). If you get
44 "database not found" or zero predictions, the video hasn't been processed yet.
45
46 See docs/EVALUATION.md for the annotation format and how to read the output.
47 ""
48
49
50 def build_parser() -> argparse.ArgumentParser:
51     p = argparse.ArgumentParser(
52         description="Evaluate detected rallies vs ground-truth annotations.",
53         epilog=_EPILOG,
54         formatter_class=argparse.RawDescriptionHelpFormatter,
55     )
56     p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end
57 per row).")
58     p.add_argument("--video", help="Path to the video file (its MD5 becomes the
59 video_id).")
60     p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of
61 --video.")
62     p.add_argument("--db", default=None, help="SQLite DB path (default: from
63 config.json).")
64     p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
65 help="Which prediction stage(s) to score (default: both).")
66     p.add_argument("--detector", default=None,
67 help="Only score candidates from this detector (e.g. yolo_hybrid,
68 tracknet_hybrid).")
69     p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
70 0.5).")
71     p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep
72 (0.1..0.9) in the report.")
73     p.add_argument("--show-failures", action="store_true",
74 help="List missed (FN) and spurious (FP) rallies with timestamps.")
75     p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to
76 this path.")
77     p.add_argument("--scaffold", action="store_true",
78 help="Write an empty annotation CSV to --annotations and exit.")
79     return p
80
81
82 def main(argv=None) -> int:
83     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
84     args = build_parser().parse_args(argv)
85
86     # Scaffold mode: just write the template and exit.
87     if args.scaffold:
88         if not args.annotations:
89             print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
90             return 2
91         write_scaffold(args.annotations)
92         print(f"Wrote empty annotation file: {args.annotations}")
93         return 0
94
95     if not args.annotations:
96         print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
97         return 2
98
99     # Resolve the video_id.
100     if args.video_id:
101         video_id = args.video_id

```

```

94         elif args.video:
95             if not os.path.exists(args.video):
96                 print(f"ERROR: video not found: {args.video}", file=sys.stderr)
97                 return 2
98             print("Computing video MD5...")
99             video_id = get_file_md5(args.video)
100             print(f"video_id = {video_id}")
101         else:
102             print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
103             return 2
104
105         db_path = args.db or _default_db_path()
106         if not os.path.exists(db_path):
107             print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
108 file=sys.stderr)
109             return 2
110
111         gt = load_annotations(args.annotations)
112         if not gt:
113             print(f"WARNING: no ground-truth rallies in {args.annotations}.",
114 file=sys.stderr)
115
116         stages = list(STAGES) if args.stage == "both" else [args.stage]
117         db = Database(db_path)
118         report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
119 detector=args.detector, with_pr_curve=args.pr_curve)
120
121         print(format_report_text(report, show_failures=args.show_failures))
122
123         if args.json_out:
124             with open(args.json_out, "w") as f:
125                 json.dump(report_to_dict(report), f, indent=2)
126                 print(f"\nJSON report written to {args.json_out}")
127
128         return 0
129
130     if __name__ == "__main__":
131         raise SystemExit(main())

```

30. user (2026-06-16T02:25:29.654Z)

```

eval_baselines\heuristic_lovo_n6.json
eval_baselines\gemini_wrapper_2026-06-10.json
eval_baselines\experiment_leaderboard.json

```

31. user (2026-06-16T02:25:29.956Z)

```
output\human_lovo_manifest.json
```

32. assistant (2026-06-16T02:25:33.819Z)

There ARE committed `eval\_baselines/\*.json` fixtures. Let me read those, the experiment engine, calibrate_wasb, distill_local, and check whether `eval\_baselines/` is gitignored or committed. Let me also look at the experiment.py module and how `VideoCandidates`'s `compare` work, plus check `github` CI wiring.

33. user (2026-06-16T02:25:35.242Z)

```

1      {
2          "name": "heuristic_velocity_windowing_LOVO",
3          "description": "The honest cross-video floor the Gen-0 learned model must beat:
calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",

```



```

4      "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local
--manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
5      "mean_f1": 0.480,
6      "std": 0.242,
7      "per_video": {
8          "adarsh_avi_singles": 0.751,
9          "kushagra_singles": 0.157,
10         "largetest_doubles": 0.737,
11         "gbaaddy": 0.309,
12         "testlarge_short": 0.267,
13         "mahadevpura_singles": 0.657
14     }
15 }
16

```

34. user (2026-06-16T02:25:36.038Z)

```

1  [
2      {
3          "timestamp": "2026-06-14T06:23:51.507393+00:00",
4          "iou": 0.5,
5          "label_set_hash": "50d98ffbc370",
6          "num_videos": 6,
7          "variant_a": {
8              "name": "shuttle_only",
9              "spec_hash": "47c867c36f2f",
10             "aggregate": {
11                 "precision": 0.31807603780927995,
12                 "recall": 0.3101947096042718,
13                 "f1": 0.3068611057861412,
14                 "mean_iou": 0.5791831889148886
15             }
16         },
17         "variant_b": {
18             "name": "shuttle_person_fusion",
19             "spec_hash": "253f4709b4c4",
20             "aggregate": {
21                 "precision": 0.3617228001510626,
22                 "recall": 0.2708939086387243,
23                 "f1": 0.2863398846047612,
24                 "mean_iou": 0.5690111885714347
25             }
26         },
27         "delta": {
28             "precision": 0.04364676234178266,
29             "recall": -0.0393008009655475,
30             "f1": -0.02052122118138,
31             "mean_iou": -0.01017200034345389
32         },
33         "honest_delta_f1": {
34             "mean_delta": -0.02052122118137999,
35             "ci_low": -0.16462806145542527,
36             "ci_high": 0.16224446866060574,
37             "n": 6.0,
38             "ci_excludes_zero": false,
39             "sign_test": {
40                 "wins": 2.0,
41                 "losses": 4.0,
42                 "ties": 0.0,
43                 "n_effective": 6.0,
44                 "p_value": 0.6875
45             }
46         }
47     },

```

```

48     {
49         "timestamp": "2026-06-14T06:24:56.487711+00:00",
50         "iou": 0.5,
51         "label_set_hash": "50d98ffbc370",
52         "num_videos": 6,
53         "variant_a": {
54             "name": "shuttle_person",
55             "spec_hash": "ee89f1e41604",
56             "aggregate": {
57                 "precision": 0.3617228001510626,
58                 "recall": 0.2708939086387243,
59                 "f1": 0.2863398846047612,
60                 "mean_iou": 0.5690111885714347
61             }
62         },
63         "variant_b": {
64             "name": "shuttle_person_audio",
65             "spec_hash": "d68c39eba87c",
66             "aggregate": {
67                 "precision": 0.4530798177857001,
68                 "recall": 0.3408138120839734,
69                 "f1": 0.3655826552797436,
70                 "mean_iou": 0.7074802852140344
71             }
72         },
73         "delta": {
74             "precision": 0.0913570176346375,
75             "recall": 0.06991990344524907,
76             "f1": 0.07924277067498242,
77             "mean_iou": 0.13846909664259965
78         },
79         "honest_delta_f1": {
80             "mean_delta": 0.07924277067498244,
81             "ci_low": -0.053508315003560535,
82             "ci_high": 0.2029045239922802,
83             "n": 6.0,
84             "ci_excludes_zero": false,
85             "sign_test": {
86                 "wins": 4.0,
87                 "losses": 2.0,
88                 "ties": 0.0,
89                 "n_effective": 6.0,
90                 "p_value": 0.6875
91             }
92         }
93     },
94     {
95         "timestamp": "2026-06-14T00:00:00+00:00",
96         "iou": 0.5,
97         "label_set_hash": "50d98ffbc370",
98         "num_videos": 6,
99         "promotion": {
100             "cue": "gate_a_net_crossings",
101             "promoted_to": "served_default",
102             "decision": "PROMOTED ? Gate A (net_crossings>=2) flipped to the live served
default",
103             "reverted": "reverted (eval?serve): the #146 served flip was REVERTED ? on the
served COARSE windowing Gate A is neutral/negative (mean served ?F1 -0.008, ~12 real rallies
dropped), confirmed by the #148 served litmus + an independent served check. The +0.074 below is
real only on the harness's recall-oriented PROPOSAL windowing; it does NOT transfer to the
served operating point. Restored config.json default_segmenter=gemini and
FusionConfig.min_crossings default=0. Gate A stays opt-in (indexing.fusion.min_crossings=2).",
104             "served_change": "config.json indexing.default_segmenter gemini->fusion_hybrid;
FusionConfig.min_crossings default 0->2",
105             "reversible": "indexing.fusion.min_crossings=0 (Gate-A rollback) or

```

```

indexing.default_segmenter=wasb_hybrid (full revert)",
106     "gate_b": "stays OFF (min_players=0, person cue OFF)",
107     "owner_approved": true,
108     "real_video_corroboration": "GX010125_1080p offline re-score: 45 candidates ->
CONTROL 45 rallies vs Gate-A 27 (dropped 18 low-crossing held-shuttle/feed windows, ~1.7min)",
109     "over_seg_segment_count_ratio": {"before": 1.68, "after": 1.01},
110     "source_archive": "docs/archives/quality-iterations/2026-06-first-ab-
experiment/README.md"
111 },
112 "variant_a": {
113     "name": "control_shuttle_proposals",
114     "spec_hash": "control",
115     "aggregate": {}
116 },
117 "variant_b": {
118     "name": "gate_a_min_crossings_2",
119     "spec_hash": "gateA_mc2",
120     "aggregate": {}
121 },
122 "delta": {
123     "f1": 0.074
124 },
125 "honest_delta_f1": {
126     "mean_delta": 0.074,
127     "ci_low": 0.042,
128     "ci_high": 0.105,
129     "n": 6.0,
130     "ci_excludes_zero": true,
131     "sign_test": {
132         "wins": 6.0,
133         "losses": 0.0,
134         "ties": 0.0,
135         "n_effective": 6.0,
136         "p_value": 0.031
137     }
138 }
139 }
140 ]

```

35. user (2026-06-16T02:25:36.378Z)

```

1      [
2      {
3          "name": "adarsh_avi_singles",
4          "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",
5          "fps": 59.94, "frame_width": 1920,
6          "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh
and Avi.rallies.csv"
7      },
8      {
9          "name": "kushagra_singles",
10         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/kushagra_traj.csv",
11         "fps": 59.94, "frame_width": 1920,
12         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/KushagraYashAviFirstVideo.rallies.csv"
13     },
14     {
15         "name": "largetest_doubles",
16         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/largetestvideo_traj.csv",
17         "fps": 29.97, "frame_width": 1920,
18         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/LargeTestVideo.rallies.csv"

```

```

19     },
20     {
21         "name": "gbaaddy",
22         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/gbaaddy_traj.csv",
23         "fps": 59.94, "frame_width": 1920,
24         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv"
25     },
26     {
27         "name": "testlarge_short",
28         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/testlarge_traj.csv",
29         "fps": 29.97, "frame_width": 1920,
30         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/TestLargeVideo.rallies.csv"
31     },
32     {
33         "name": "mahadevpura_singles",
34         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura_traj.csv",
35         "fps": 29.97, "frame_width": 1920,
36         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/MahadevpuraSingles.rallies.csv"
37     }
38 ]
39

```

36. user (2026-06-16T02:25:38.179Z)

```

eval_baselines/experiment_leaderboard.json
eval_baselines/gemini_wrapper_2026-06-10.json
eval_baselines/heuristic_lovo_n6.json
---CHECK-IGNORE---
output/human_lovo_manifest.json

```

37. assistant (2026-06-16T02:25:42.838Z)

Key finding: `eval\_baselines/\*.json` ARE committed (tracked); `output/human\_lovo\_manifest.json` IS gitignored (the `git check-ignore` matched it). The manifest points to absolute paths OUTSIDE the repo (`C:/Users/avidu/Projects/Annotation Setup/...`) for both trajectories and labels. So the golden trajectory CSVs and `.rallies.csv` label files live outside the repo and are NOT committed.

Now let me read the experiment engine and the two remaining dependencies (calibrate_wasb, distill_local) to complete the artifact/format trace, plus check the regression baseline path and CI wiring.

38. user (2026-06-16T02:25:44.205Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`           ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`          ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`        ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine`     ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`        ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`       ? `pct_improvement`

```

```

15
16 The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17 (per-frame signals ? run once on the GPU/WSL box, persisted into
18 `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19 signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20 given persisted candidate features it produces `List[Interval]` deterministically,
21 with no GPU and zero production risk. So most experiments are pure offline
22 re-scoring of stored data and never touch the served pipeline.
23
24 Three modes (`EXPERIMENT_HARNESS.md` §5):
25 - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26   aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27   held-out loop, but variant-parameterised.
28 - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29   variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30   human spot-checks (and what two highlight reels would show side by side).
31 - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32   (exit 0/1/3); **rollback = flip the variant/config, never `git revert`**
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Callable, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count
56 ratio
57 scores
58 from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
59
60 logger = logging.getLogger(__name__)
61
62 # Where a comparison run appends one reproducible record. Tracked location (NOT under
63 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
64 # regression.DEFAULT_BASELINE_PATH.
65 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
66 _METRICS = ("precision", "recall", "f1", "mean_iou")
67
68 class ExperimentError(ValueError):
69     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70     score an unlabeled video with no pinned model, or a gate referencing an absent
71     feature)."""
72
73 # ----- Variant
74
75 @dataclass
76 class Variant:
77

```

```

78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the control behaviour (no gate, no learned
82     filter), so a variant that merely carries a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91       ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95       whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for ``compare_unlabeled`` on a learned variant.
100     - ``feature_names`` ? the persisted features the learned filter consumes. When set
101       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102       (honest) ? the recipe, not a pinned artifact.
103     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104       overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105     """
106
107     name: str
108     segmenter: str = "wasb_hybrid"
109     config_overrides: Dict[str, Any] = field(default_factory=dict)
110     cue_flags: Dict[str, bool] = field(default_factory=dict)
111     min_crossings: int = 0
112     min_players: int = 0
113     classifier_model: Optional[str] = None
114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,

```

```

138         "merge_gap": self.merge_gap,
139         "tune_threshold": self.tune_threshold,
140         "calibrate": self.calibrate,
141     }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
151         the
152         same, so a regression is always traceable to a recipe change, never to drift."""
153         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154         return sha1(blob.encode()).hexdigest()[:12]
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183         col = vc.features.get(name)
184         n = len(vc.intervals)
185         if col is None:
186             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                            name, vc.name)
188             return [0.0] * n
189         if len(col) != n:
190             raise ExperimentError(
191                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192         return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196         cols = [_feature_column(vc, name) for name in feature_names]
197         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:

```

```

201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
202     n = len(vc.intervals)
203     mask = [True] * n
204     if variant.min_crossings > 0:
205         col = _feature_column(vc, "net_crossings")
206         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
207     if variant.min_players > 0:
208         col = _feature_column(vc, "players_in_court")
209         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
210     return mask
211
212
213 def predict(variant: Variant, vc: VideoCandidates,
214            model: Optional[LogisticRegression] = None,
215            threshold: Optional[float] = None,
216            calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217     """Variant prediction: gate by persisted features, optionally filter by a learned
218     model, then merge. Pure and deterministic.
219
220     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225     them.
226
227     """
228     mask = _gate_mask(variant, vc)
229     if variant.feature_names:
230         if model is None:
231             if variant.classifier_model is None:
232                 raise ExperimentError(
233                     f"variant {variant.name!r} is learned but has no model to predict "
234                     f"with (pass a fitted model or set classifier_model)")
235             model = LogisticRegression.load(variant.classifier_model)
236             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237 variant.feature_names))]
238         if calibrator is not None:
239             proba = [calibrator(p) for p in proba]
240             thr = variant.threshold if threshold is None else threshold
241             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
242         kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
243         return merge_overlaps(kept, gap_tol=variant.merge_gap)
244
245
246 def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
247                        train_videos: Sequence[VideoCandidates], iou: float
248                        ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
249     """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
250     (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
251     (use the variant default). Honest: never looks at the held-out video.
252     """
253     calibrator: Optional[Callable[[float], float]] = None
254     if variant.calibrate:
255         raw: List[float] = []
256         y: List[int] = []
257         for vc in train_videos:
258             raw.extend(float(p) for p in
259 model.predict_proba(_feature_matrix(vc, variant.feature_names or
260 [])))
261             y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
262         if variant.calibrate == "platt":
263             calibrator = fit_platt(raw, y)
264         elif variant.calibrate == "isotonic":

```



```

261         calibrator = fit_isotonic(raw, y)
262     else:
263         raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
264     threshold: Optional[float] = None
265     if variant.tune_threshold:
266         best_t, best_f1 = variant.threshold, -1.0
267         for k in range(1, 20): # grid 0.05..0.95 on the train
fold's rally F1
268             t = round(0.05 * k, 2)
269             f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                        vc.gts or [], iou).f1 for vc in train_videos]
271             mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272             if mean_f1 > best_f1:
273                 best_f1, best_t = mean_f1, t
274             threshold = best_t
275     return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                        iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310
311         """
312         for vc in videos:
313             if vc.gts is None:
314                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
315
316         per_video: List[Dict[str, Any]] = []

```

```

316     predictions: Dict[str, List[Interval]] = {}
317
318     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319     pinned_model = (LogisticRegression.load(variant.classifier_model)
320                     if variant.classifier_model is not None else None)
321     all_model = None
322     if recipe_lovo and not honest:
323         all_model = _train(variant, videos, iou)
324
325     for i, vc in enumerate(videos):
326         cal: Optional[Callable[[float], float]] = None
327         thr: Optional[float] = None
328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():

```

```

371         gts = gts_by_name.get(name, [])
372         got = _segm.f1_at_overlaps(preds, gts, overlaps)
373         for ov in overlaps:
374             f1k[ov].append(got[ov])
375         ratios.append(_segm.segment_count_ratio(preds, gts))
376
377     def _mean(xs: List[float]) -> float:
378         return sum(xs) / len(xs) if xs else 0.0
379     out: Dict[str, Any] = {"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380     out["segment_count_ratio"] = _mean(ratios)
381     return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404               iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422

```

```

423     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427     per_video_delta = [
428         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429         for p in eval_b["per_video"]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455         agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465           the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
a
468         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469         ``metrics.match_intervals`` ? no new matching logic.
470         """
471         preds_a = predict(variant_a, video)
472         preds_b = predict(variant_b, video)
473         mr = match_intervals(preds_a, preds_b, agree_iou)
474
475         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
476                 for m in mr.matches]
477         agree_iou = [m.iou for m in mr.matches]
478         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
479         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
480
481     def _dur(ivs: List[Interval]) -> float:
482         return sum(e - s for s, e in ivs)
483

```

```

484     return {
485         "video": video.name,
486         "agree_iou": agree_iou,
487         "variant_a": variant_a.name,
488         "variant_b": variant_b.name,
489         "num_a": len(preds_a),
490         "num_b": len(preds_b),
491         "num_agreed": len(agreed),
492         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
493         "duration_a": _dur(preds_a),
494         "duration_b": _dur(preds_b),
495         "agreed": agreed,
496         "a_only": a_only,
497         "b_only": b_only,
498     }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                          gts: Optional[List[Interval]] = None,
545                          feature_names: Sequence[str] = ()) -> VideoCandidates:
546     """Bridge persisted detection ? `VideoCandidates` via
547     `Database.query_candidate_segments`.

```

```

547
548     ``feature_names`` selects which persisted features to lift out of each row's
``stats``
549     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550     function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551     but works against any `Database` (including an in-memory one in tests).
552     """
553     rows = db.query_candidate_segments(video_id, detector=detector)
554     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555     features: Dict[str, List[float]] = {}
556     for name in feature_names:
557         if name == "duration":
558             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559         else:
560             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561     return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

39. user (2026-06-16T02:25:44.398Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10     filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11     active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12     -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21     python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37     from backend.eval import windowing as _windowing
38

```

```

39     Interval = Tuple[float, float]
40
41     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
42 "net_crossings"]
43     # Recall-oriented proposal: deliberately permissive so real rallies are among the
44     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
45     # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
46     that
47     # module's docstring for the eval?serve bug these named presets close. The bare tuples
48     are
49     # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
50     # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
51     PROPOSAL = _windowing.PROPOSAL.as_tuple()
52     BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
53     learning
54
55     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
56                         proposal: Tuple[float, float, float] = PROPOSAL) ->
57     Tuple[List[Interval], List[List[float]]]:
58     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
59     rows)."""
60     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
61     vt, mg, mwd = proposal
62     wins = TrackNetRunner.trajectory_to_action_windows(
63         points, fps=fps, frame_width=frame_width,
64         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
65     intervals = [(w["start"], w["end"]) for w in wins]
66     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
67     crossings
68     X: List[List[float]] = []
69     for w, g in zip(wins, gated):
70         dur = max(1e-6, w["end"] - w["start"])
71         win_frames = max(1.0, dur * fps)
72         X.append([
73             dur, # rally length (sec) ?
74             float(w["peak_velocity"]), # already normalized by
75             float(w["mean_velocity"]),
76             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
77             float(g.crossings), # net crossings (count) ?
78             the rally signal
79         ])
80     return intervals, X
81
82     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
83     List[Interval]:
84     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
85     vt, mg, mwd = BASELINE_LOCAL
86     wins = TrackNetRunner.trajectory_to_action_windows(
87         points, fps=fps, frame_width=frame_width,
88         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
89     return [(w["start"], w["end"]) for w in wins]
90
91     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
92     fps, fw = float(spec["fps"]), float(spec["frame_width"])
93     intervals, X = build_candidates(spec["trajectory"], fps, fw)
94     gts = load_golden_gts(spec["labels"])
95     y = label_candidates(intervals, gts, iou)
96     return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
97     X,

```

```

91         "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}}
92
93
94     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
95         threshold: float = 0.5) -> List[Interval]:
96         if not intervals:
97             return []
98         proba = model.predict_proba(X)
99         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
100         return merge_overlaps(pos, gap_tol=1.0)
101
102
103     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
104         model_out: Optional[str] = None) -> dict:
105         videos = [load_video(s, iou) for s in specs]
106         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
107             f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
108         for v in videos:
109             ceil = score(v["intervals"], v["gts"], iou).recall
110             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
111                 f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
112
113         result: dict = {"videos": [v["name"] for v in videos]}
114         if len(videos) >= 2:
115             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
116             clf, base = [], []
117             for i, held in enumerate(videos):
118                 others = [v for j, v in enumerate(videos) if j != i]
119                 X = [row for v in others for row in v["X"]]
120                 y = [t for v in others for t in v["y"]]
121                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
122                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
123                 sc = score(preds, held["gts"], iou)
124                 bsc = score(held["baseline"], held["gts"], iou)
125                 clf.append(sc)
126                 base.append(bsc)
127                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}")
128                 f"  | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
129             def mean(xs, a):
130                 return sum(getattr(s, a) for s in xs) / len(xs)
131             print(f"  MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
132                 f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
133                 f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
134             result["mean_learned_f1"] = mean(clf, "f1")
135             result["mean_baseline_f1"] = mean(base, "f1")
136
137             # Final model on ALL videos (for production / running on a fresh held-out video).
138             Xall = [row for v in videos for row in v["X"]]
139             yall = [t for v in videos for t in v["y"]]
140             final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141             assert final.w is not None # set by fit()
142             weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143             print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144             if model_out:
145                 final.save(model_out)
146                 print(f"final model (all videos) -> {model_out}")
147                 result["model_out"] = model_out
148             return result
149

```



```

150
151     def main() -> None:
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166

```

40. assistant (2026-06-16T02:25:47.490Z)

Now I have a near-complete picture. Let me read `calibrate\_wasb.py` (the `load\_golden\_gts` / `make\_predict\_fn` source) and check the CI workflow + how regression is invoked there, plus confirm whether a committed `regression\_baseline.json` exists.

41. user (2026-06-16T02:25:48.448Z)

```

1     """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3     Wires the already-tested pieces together:
4         WASB trajectory CSV (Frame,Visibility,X,Y)
5         -> TrackNetRunner.trajectory_to_action_windows(cfg)    [predict_fn]
6         -> backend.eval.calibration (improvement_report / cross_validate)
7
8     Run (after a full-video WASB pass exists):
9         python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10            --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12     Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13     same GT is an **optimistic** upper bound. The trustworthy number is the
14     cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15     `leave_one_video_out` for precision/F1 generalization).
16     """
17     from __future__ import annotations
18
19     import argparse
20     from typing import List, Tuple, Callable
21
22     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
23     from backend.eval.calibration import (
24         select_best, improvement_report, cross_validate, evaluate,
25     )
26
27     Interval = Tuple[float, float]
28     Config = Tuple[float, float, float]    # (velocity_thresh, merge_gap,
min_window_duration)
29
30     BASELINE: Config = (0.02, 2.0, 2.0)    # the WasbConfig / trajectory_to_action_windows
defaults
31
32
33     def load_golden_gts(csv_path: str) -> List[Interval]:

```

```

34     """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35
36     Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37     start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38     driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39     """
40     from backend.eval.gt_loader import load_gt_intervals
41     return load_gt_intervals(csv_path, strict=False)
42
43
44     def make_predict_fn(trajecory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
47
48         def predict_fn(cfg: Config) -> List[Interval]:
49             vt, mg, mwd = cfg
50             wins = TrackNetRunner.trajectory_to_action_windows(
51                 points, fps=fps, frame_width=frame_width,
52                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53             )
54             return [(w["start"], w["end"]) for w in wins]
55
56         return predict_fn
57
58
59     def default_grid() -> List[Config]:
60         return [(vt, mg, mwd)
61                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62                 for mg in (1.0, 2.0, 3.0)
63                 for mwd in (1.0, 2.0, 3.0)]
64
65
66     def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
67            iou: float = 0.5) -> dict:
68         gts = load_golden_gts(labels_csv)
69         if not gts:
70             raise SystemExit(f"no usable golden rallies in {labels_csv}")
71         predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
72         grid = default_grid()
73
74         base = evaluate(predict_fn(BASELINE), gts, iou)
75         best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
76         fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)    # OPTIMISTIC
(fit on full GT)
77         cv = cross_validate(predict_fn, grid, gts, k=5)                      # HONEST held-
out recall
78
79         print("=== WASB rally-segmentation calibration ===")
80         print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
81         print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
82               f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
83         print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
84         print(f" precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
85               f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
86         print(f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
87               f"({fit['f1_delta_pct']:+.1f}%)")
88         print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
89               f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
90               f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
91         print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")

```

```

92         return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
93
94
95     def main():
96         ap = argparse.ArgumentParser()
97         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
98         ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
99         ap.add_argument("--fps", type=float, default=59.94)
100        ap.add_argument("--frame-width", type=float, default=1920.0)
101        ap.add_argument("--iou", type=float, default=0.5)
102        args = ap.parse_args()
103        run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
104
105
106    if __name__ == "__main__":
107        main()
108

```

42. user (2026-06-16T02:25:49.017Z)

.github\workflows\ci.yml

43. user (2026-06-16T02:25:49.492Z)

No files found

44. user (2026-06-16T02:25:50.962Z)

---ALL TRACKED CSV under tests---

45. assistant (2026-06-16T02:25:55.438Z)

No committed golden CSVs, no committed `regression\_baseline.json`, no tracked CSV/JSON fixtures under `tests/`. Tests build CSVs inline via `tmp\_path`. Let me verify the CI workflow to see how (if at all) regression/eval is invoked there, and confirm the `output/` and `eval\_baselines/` gitignore behavior precisely.

46. user (2026-06-16T02:25:56.441Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11    concurrency:
12        group: ci-${{ github.ref }}
13        cancel-in-progress: true
14
15    jobs:
16        # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17        # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18        # tracked .py, including scripts/ and training/, which the backend-only
19        # py_compile smoke sweep does not reach.
20        lint:
21            name: Ruff lint
22            runs-on: ubuntu-latest
23            steps:

```

```

24     - uses: actions/checkout@v6
25     - uses: actions/setup-python@v6
26     with:
27         python-version: "3.13"
28     - name: Install ruff
29       run: python -m pip install ruff
30     - name: Ruff check
31       run: ruff check . --output-format github
32
33     # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34     # per-module quarantine list ? ratchet by removing entries as modules clean
35     # up. Typed core deps are installed so mypy sees their real signatures
36     # (without them ignore_missing_imports would Any-stub half the checks).
37     typecheck:
38       name: Mypy (lenient baseline)
39       runs-on: ubuntu-latest
40       steps:
41         - uses: actions/checkout@v6
42         - uses: actions/setup-python@v6
43         with:
44             python-version: "3.13"
45         - name: Install mypy + typed core deps
46           run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47         - name: Mypy
48           run: mypy backend training
49
50     # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
51     # module-level import error shipping while the suite stays green.
52     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53     # on machines without the GPU stack (test_import_smoke stubs what's absent).
54     import-smoke:
55       name: Syntax sweep + import smoke (no GPU stack)
56       runs-on: ubuntu-latest
57       steps:
58         - uses: actions/checkout@v6
59         - uses: actions/setup-python@v6
60         with:
61             python-version: "3.13"
62         - name: Install minimal deps
63           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64         - name: Run import smoke tests
65           run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68       name: Full test suite (offline, mocked)
69       runs-on: ubuntu-latest
70       steps:
71         - uses: actions/checkout@v6
72         - uses: actions/setup-python@v6
73         with:
74             python-version: "3.13"
75             cache: pip
76         - name: System libs for opencv-python
77           run: |
78             sudo apt-get update
79             sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80         - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81           # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82           # need an out-of-band index that PEP 621 can't express. Keep installing
83           # them separately here, before the editable install pulls the rest.
84           run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu

```

```

85     - name: Install package (editable) + dev tooling
86       # Editable PEP 621 install: runtime deps + the `dev` group
87       # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88       run: python -m pip install -e .[dev]
89       # obsreport (private, soft dep) is absent here: reporting tests skip via
90       # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91       # is therefore calibrated against THIS lane's number (local runs with
92       # obsreport installed measure a little higher).
93       # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94       # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95       # without an owner decision.
96     - name: Run full suite (with coverage floor)
97       run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98

```

47. user (2026-06-16T02:25:59.493Z)

```

eval_baselines/gemini_wrapper_2026-06-10.json
---gitignore eval/output rules---
.gitignore:39:output/    output/foo.json
---grep gitignore---
.gitignore
binary file matches (found "\0" byte around offset 716)

```

48. assistant (2026-06-16T02:26:05.463Z)

I have a complete picture now. Key confirmations:

- CI (`.ci.yml`) runs lint, mypy, import-smoke, and the full pytest suite with a 70% coverage floor. It does NOT invoke `run\_regression.py` or `run\_eval.py` as a gate ? the regression machinery is exercised only via pytest (`.test\_eval\_regression.py`) using inline `tmp\_path` CSVs and in-memory DBs.
- `output/` is gitignored (line 39). `eval\_baselines/` is NOT gitignored ? its JSONs are committed.
- No committed `regression\_baseline.json`, no committed golden `rallies.csv`, no committed `\*\_golden\_features.json`. The golden labels/trajectories live at absolute paths outside the repo per the gitignored `output/human\_lovo\_manifest.json`.

Let me quickly confirm the `.gitignore` line 39 context and whether `eval\_baselines/` or any golden fixture is explicitly allowed/ignored, then I have everything.

49. user (2026-06-16T02:26:06.948Z)

```

35:# SQLite databases & Local output files
36:*.db
37:*.db-journal
38:*.db-wal
39:output/
40:output_dir/
46:Thumbs.db
60:# Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from
62:scratch/

```

50. assistant (2026-06-16T02:26:10.457Z)

I have everything needed. `output/` is gitignored (so the manifest, golden feature JSONs, the runtime DB, and `regression\_baseline.json` default path all live in untracked territory), while `eval\_baselines/` is tracked and holds the committed structured fixtures. Let me compile the structured findings.

51. user (2026-06-16T02:27:49.842Z)

Structured output provided successfully

52. assistant (2026-06-16T02:28:01.302Z)

I've completed the mapping and returned the structured findings. Summary of what I found:

****The eval stack already exists and is largely video-free at the scoring boundary**** ? but no flow currently runs from committed in-repo files end-to-end. The closest existing video-free, file-driven engine is `backend/eval/ablation.py` + `backend/eval/experiment.py`, which run 100% on CPU from `\_golden\_features.json` (intervals + feature columns + golden gts) with no DB, no video, no Gemini. The problem is those JSONs are written to the ****gitignored `output/`**** dir, and the golden corpus (trajectories + `.rallies.csv`) lives at absolute paths ****outside the repo**** via the gitignored `output/human\_lovo\_manifest.json`.

****Committed structured fixtures that DO survive a clean checkout**** all live in the tracked `eval\_baselines/` dir: `heuristic\_lovo\_n6.json`, `experiment\_leaderboard.json`, `gemini\_wrapper\_2026-06-10.json`. There is ****no committed `regression\_baseline.json`**** (the default path is reserved but the file doesn't exist), ****no committed golden label CSVs****, and ****no test fixtures**** (tests build CSVs inline via `tmp\_path`).

****For the design****: reuse `metrics`, `segmentation\_metrics`, `gt\_loader` (the one canonical CSV reader accepting both golden conventions), `regression` (baseline store + gt_hash + exit 0/1/3 gate), `experiment.VideoCandidates`/`evaluate\_variant`, `ablation.load\_videos`, and `splits.assert\_disjoint`. What must be ADDED: (a) a committed small golden fixture (labels + frozen predictions/feature substrate, repo-relative), (b) a thin "predictions-from-file" loader so `regression` runs without the SQLite DB, (c) a repo-relative manifest, and (d) optionally wiring regression into CI (today it's pytest-only, not a gate).

One note: `git pull --ff-only` failed with "Cannot fast-forward to multiple branches" ? I proceeded read-only on local `master` (clean vs `origin/master`), which is fine for this mapping task.